# Reinforcement Learning for Soft Manipulator Control:

Reward Architecture and Curriculum Design on Cosserat Rods

Qifan Wen
Electrical and Computer Engineering
The Ohio State University
wen.679@buckeyemail.osu.edu

*Advisor:* Prof. Abhishek Gupta gupta.706@osu.edu

*Master of Science Non-Thesis Graduation Project Report*

**Abstract**

Reinforcement learning for soft and continuum robots is bottlenecked by the cost of first-principles simulation: a Cosserat rod snake robot requires ∼500 substeps per control step, costing ∼46 ms per environment transition. This report investigates two directions for enabling practical RL-based control of a soft manipulator: neural surrogate modeling and reward architecture design.

**Neural Surrogate Modeling.** A data-driven MLP trained on 4.3M PyElastica transitions achieves centre-of-mass position $R^2 = 0.852$ in single-step prediction, targeting a four-to-five order-of-magnitude inference speedup. Velocity prediction remains weak ($R^2 = 0.145$); physics-informed extensions (PINN, DD-PINN) are proposed as future work.

**Reward Design and Ablation.** Systematic RL experiments on DisMech show that reward shaping and curriculum learning dominate performance: a five-variant ablation reveals a 52% range in the fair distance metric under identical hyperparameters, and curriculum learning alone accounts for 16% improvement over the vanilla baseline. Extended training to 50M frames confirms these are reward-landscape limitations, not sample-budget issues: the $\exp(-5d)$ reward vanishes at operating distances ($d \approx 0.6$), creating a hard learning plateau.

**Normalized Reward Architecture.** A normalized multi-component reward with per-objective potential-based reward shaping (PBRS) decouples component scale from importance weights, achieving a best distance of $d = 0.543$ (−24% vs. baseline) and enabling reward-hacking detection through per-component logging.

**Evaluation and Limitations.** Deterministic evaluation of the best PPO checkpoint reveals a large train–eval gap: the training best reward of 90.20 drops to a mean of 8.68 over 20 episodes, with the manipulator plateauing at ∼0.7 m from the target. The policy learns coarse directional movement but fails to achieve close tracking, confirming the reward-landscape plateau as a genuine performance ceiling. We also document SAC gradient instability without entropy regularization, with actor gradient norms growing to $10^{10}$ over 20M frames.

# Contents

**Glossary** **5**

**1 Introduction** **7**

**2 Related Work** **7**

2.1 Neural Surrogates for ODE/PDE . . . . . 7

2.2 RL Control for Snake and Soft Robots . . . . . 8

**3 System Overview: Snake Robot Simulation** **9**

3.1 Cosserat Rod Formulation . . . . . 9

3.2 CPG Control . . . . . 11

3.3 Physics Backends . . . 12

**4 Neural Surrogate Models** **13**

4.1 Data Collection . . . 13

4.2 MLP Surrogate . . . 14

4.2.1 Input Representation . . . 14

4.2.2 Per-Element CPG Phase Encoding . . . 15

4.2.3 Architecture and Prediction . . . 15

4.2.4 Training . . . 16

4.2.5 Hyperparameters . . . 17

4.3 MLP Results . . . 17

4.4 Physics-Informed Extensions (Proposed) . . . 20

4.5 Surrogate Comparison . . . 20

4.6 Surrogate Validation: RL on Elastica (E1) . . . 20

4.6.1 Environment and Training Setup . . . 20

4.6.2 Direct Simulation Results . . . 21

4.6.3 Surrogate Environment Results . . . 22

4.6.4 Challenges . . . 23

**5 Reward Design for Soft Manipulator Control** **23**

5.1 DisMech: Task and Setup . . . 24

5.1.1 Task Description . . . 24

5.1.2 Training Configuration . . . 24

5.2 Reward Variants and Initial Ablation (E2) . . . 25

5.2.1 Reward Variants . . . 25

5.2.2 Results . . . 27

5.2.3 Discussion . . . 28

5.3 Reward Landscape Diagnostics (E3) . . . 30

5.4 Extended Training, 50M Frames (E4) . . . 31

5.5 SAC vs. PPO Stability (E5) . . . 32

5.6 Normalized Reward Architecture . . . 33

5.6.1 Problems with Ad-Hoc Design . . . 33

5.6.2 Architecture . . . 34

5.7 Validation Ablation (E6) . . . 35

5.8 Steepness and Environment Scaling (E7) . . . . . 37
5.9 Best Checkpoint Evaluation (E8) . . . . . 39

**6 Conclusion** **41**

**A Detailed Physics Derivations** **43**
A.1 Staggered Grid Discretization . . . . . 43
A.2 Method of Lines and Explicit Integration . . . . . 45
A.3 How Spatial Discretization Converts PDEs to ODEs . . . . . 45
A.3.1 The Staggered Grid Layout . . . . . 45
A.3.2 The Difference Matrix $\mathbf{D}$ (Node $\rightarrow$ Element) . . . . . 46
A.3.3 The Transpose $\mathbf{D}^T$ (Element $\rightarrow$ Node) . . . . . 46
A.3.4 Adjoint Structure and Energy Conservation . . . . . 47
A.3.5 Stage 1: Eliminating Spatial Derivatives . . . . . 47
A.3.6 Stage 2: Reduction to First-Order Form . . . . . 48
A.4 External Force Handling . . . . . 48
A.5 DisMech Formulation . . . . . 49

**B Physics-Informed Neural Network Details (Proposed)** **50**
B.1 Standard PINN . . . . . 50
B.2 Physics Regularizer . . . . . 51
B.3 DD-PINN Method . . . . . 51
B.4 Limitations of PINNs . . . . . 52
B.5 Differentiable Cosserat Rod Right-Hand Side . . . . . 52
B.6 Physics-Regularized Loss . . . . . 53
B.7 DD-PINN Loss and Training . . . . . 53
B.8 Nondimensionalization . . . . . 53
B.9 DD-PINN Configuration . . . . . 54
B.10 Four-Way Surrogate Comparison . . . . . 54

**C Issue Tracker** **55**

## Glossary

**Cosserat rod** A continuum mechanics model for slender elastic bodies that tracks both position and orientation (director frames) along the rod centreline, capturing bending, torsion, shear, and extension.

**PyElastica** An open-source Cosserat rod simulator using explicit symplectic (Position Verlet) time integration. Used in this work for data collection and direct RL training.

**DisMech** A Cosserat rod simulator using implicit backward Euler time integration. Used for the Choi2025 follow-target RL experiments.

**CPG** *Central Pattern Generator.* A bio-inspired oscillator that produces rhythmic motor commands. The snake robot's action space consists of five normalised CPG parameters (amplitude, frequency, spatial wavenumber, phase offset, lateral bias).

**Serpenoid curve** The travelling-wave body shape $\kappa(s,t) = A\sin(2\pi(ks - ft) + \phi_0)$ that produces serpentine locomotion; parameterised by CPG actions.

**Surrogate model** A neural network $f_\theta$ trained to approximate the physics simulator's state transition, replacing an expensive first-principles solver with a single GPU forward pass.

**MLP** *Multi-Layer Perceptron.* A feedforward neural network of fully connected layers. The implemented surrogate uses three hidden layers of 512 units with LayerNorm and SiLU activations.

**PINN** *Physics-Informed Neural Network.* A neural network whose training loss includes a residual term derived from the governing PDEs, encouraging physics-consistent predictions. Proposed but not implemented in this work.

**DD-PINN** *Domain-Decoupled Physics-Informed Neural Network.* A PINN variant where the network outputs parameters of an analytically differentiable ansatz rather than the state itself, eliminating autodiff cost and satisfying initial conditions exactly. Proposed but not implemented in this work.

**RFT** *Resistive Force Theory.* An anisotropic friction model that decomposes ground contact forces into tangential and normal components, widely used for limbless locomotion on flat surfaces.

**RL** *Reinforcement Learning.* A learning paradigm where an agent interacts with an environment to maximise cumulative reward.

**PPO** *Proximal Policy Optimisation.* An on-policy actor-critic algorithm that constrains policy updates via a clipped surrogate objective, balancing sample efficiency with training stability.

**SAC** *Soft Actor-Critic* [1]. An off-policy actor-critic algorithm that maximises a combined reward-plus-entropy objective, encouraging exploration and robust policies.

**GAE** *Generalised Advantage Estimation.* A variance-reduction technique for policy gradient methods that exponentially weights multi-step temporal-difference errors via a parameter $\lambda$.

**PBRS** *Potential-Based Reward Shaping.* A reward augmentation $F(s, s') = \gamma\,\Phi(s') - \Phi(s)$ that preserves optimal-policy invariance while providing denser learning signal. Used per-objective in the normalised reward architecture.

**Curriculum learning** A training strategy that gradually increases task difficulty. In this work, the target spawn distance starts close and expands over training, ensuring the agent encounters reward-dense regions early.

**HRL** *Hierarchical Reinforcement Learning.* A framework that decomposes control into multiple levels (e.g. meta-controller, approach policy, coil policy) to handle temporally extended tasks.

**TorchRL** A PyTorch-native RL library providing composable environment wrappers, replay buffers, and loss modules. Used as the training framework throughout this project.

**CoM** *Centre of Mass.* The mass-weighted average position of all discretisation nodes; used as a summary statistic for locomotion evaluation.

**CFL condition** *Courant–Friedrichs–Lewy condition.* A numerical stability constraint relating time-step size to spatial resolution and wave speed in explicit integrators.

# 1 Introduction

Reinforcement learning for soft and continuum robots is bottlenecked by the cost of first-principles simulation. A snake robot modeled as a Cosserat rod requires resolving elastic body dynamics, anisotropic ground friction, and internally generated muscle moments at every simulation substep — PyElastica [2] uses 500 Position Verlet substeps per control step, costing $\sim$46 ms per environment step. Comprehensive surveys of soft robot modeling [3] and model-based control [4] highlight that this computational burden remains a central bottleneck. This report presents a neural surrogate $f_\theta \approx T$ that replaces the transition operator with a single GPU forward pass ($\sim$0.001 ms), targeting a four-to-five order-of-magnitude speedup for RL training.

The project has two main contributions:

1. **Neural surrogate modeling** — a data-driven MLP surrogate trained on 4.3M Elastica transitions, with physics-regularized and DD-PINN extensions proposed as future directions (Section 4). A direct-simulation RL baseline on Elastica validates the environment pipeline (Section 4.6).

2. **Reward design for soft manipulator control** — systematic RL experiments on DisMech reveal that reward function and curriculum learning dominate performance; a normalized multi-component architecture with per-objective PBRS addresses the identified structural problems (Section 5).

Two physics backends are investigated — PyElastica (explicit symplectic) and DisMech (implicit backward Euler). The shared simulation framework is described in Section 3.

# 2 Related Work

## 2.1 Neural Surrogates for ODE/PDE

Table 1 summarizes neural surrogate methods directly relevant to the Cosserat rod setting.

Table 1: Neural surrogates for soft robot / Cosserat rod dynamics.

| Reference | Method | Architecture | Speedup | Accuracy | Key contribution |
|---|---|---|---|---|---|
| DD-PINN [5] | Domain-decoupled PINN | Feedforward NN | 44,000× | 3 mm EE error (2.3%) | Time domain decoupled from network; closed-form gradients; MPC at 70 Hz on GPU |
| KNODE-Cosserat [6] | Hybrid physics + Neural ODE | Neural ODE residual | — | 58.7% vs. physics-only | Coarse Cosserat backbone + learned correction; trained on PyElastica data |
| SoRoLEX [7] | Learned simulator for RL | LSTM | ∼10× | Task-space obs. | Full pipeline: sim data → surrogate → PPO; JAX GPU parallelization |
| PINN Soft Robot [8] | Physics-informed NN | PINN | 467× | — | MPC at 47 Hz; as little as 1 training dataset; articulated soft robot |
| **This work** | Data-driven MLP surrogate | MLP (3×512) | ∼46,000× (proj.) | $R^2$=0.393 overall; CoM $R^2$=0.852 | Per-element CPG phase encoding; single-step MSE training; full 124-dim state |

The most closely related prior work is SoRoLEX, which shares the same pipeline structure (collect trajectories, train neural dynamics model, run RL in learned environment). This work differs in architecture (MLP vs. LSTM), prediction target (full 124-dimensional Cosserat rod state vs. task-space observation), and the explicit use of a multi-step rollout loss.

## 2.2 RL Control for Snake and Soft Robots

Table 2 consolidates RL approaches for snake and soft robot control.

Table 2: RL methods for snake and soft robot control.

| Reference | Robot | Actions | Algo. | Simulator | Key contribution |
|---|---|---|---|---|---|
| Shi et al. [9] | 3-link kinem. | Joint vel. (disc.) | DQN | Analytical | SE(2) fiber bundle reduces state 5D→3D |
| Liu et al. [10] | 4-link soft | CPG + option (6) | PPO | Custom | Multi-agent game for contact-aware nav. |
| Liu et al. [11] | 9-link wheeled | Gait offset (1) | PPO | MuJoCo | RL modulates undulation for path following |
| Liu et al. [12] | 4-link soft | CPG tonic (4) | PPO | MuJoCo | 12-level curriculum + domain randomization |
| Zheng et al. [13] | 7-link rigid | CPG (5) | PPO | MuJoCo | Two-phase curriculum; series elastic act. |
| Jiang et al. [14] | 11-joint CO-BRA | Dual CPG (7) | DDPG | MuJoCo | A*→RL→CPG→PID hierarchy |
| Schaffer et al. [15] | Biohybrid worm | Muscle act. (42) | PPO | PyElastica | Adaptive muscle force ceilings |
| Choi and Tong [16] | Soft manip. | $\Delta\kappa$ (5) | SAC [1] | DisMech | Implicit Euler for contact-rich tasks |
| Bing et al. [17] | Soft snake | CPG (5) | PPO | PyElastica | CPG reduces exploration vs. per-joint |
| Hong et al. [18] | Soft robot | Joint torques | PPO | FEM | FEM essential for sim-to-real fidelity |
| Janner et al. [19] | Rigid artic. | Joint torques | SAC+M | MuJoCo | Model-based short-horizon planning |
| Sanchez-Gonzalez et al. [20] | Particles | Forces | — | Custom | Graph network learns mesh-based physics |
| **This work** | Soft snake | CPG (5) | PPO | Elast./DM | Surrogate for 4–5 OOM speedup |

Key themes: CPG-based action spaces dominate snake robot RL, reducing exploration burden relative to per-joint torque control. Curriculum learning and domain randomization bridge the sim-to-real gap. All approaches share the simulation cost bottleneck motivating this work: MuJoCo costs $\sim$1 ms/step, while Cosserat rod simulators cost 12–46 ms/step.

# 3 System Overview: Snake Robot Simulation

This section describes the shared simulation framework underlying all experiments: the Cosserat rod formulation, CPG actuation, and the two physics backends.

## 3.1 Cosserat Rod Formulation

The system is fully specified by the tuple $\mathcal{S} = (\mathcal{L},\ \Omega,\ \mathrm{BC},\ \mathrm{IC},\ \boldsymbol{\theta})$:

Table 3: Well-posedness tuple for the Elastica Cosserat rod system.

| Component | Name | Specification |
|---|---|---|
| $\mathcal{L}$ | Governing equations | Cosserat rod PDEs: conservation of linear and angular momentum with linear elastic constitutive law |
| $\Omega$ | Domain | $s \in [0, L]$, $L = 0.5\,\mathrm{m}$ (arc length) $\times\ t \in [0, T]$ (time) |
| BC | Boundary conditions | Free–free: $\mathbf{F} = \mathbf{M} = \mathbf{0}$ at $s = 0$ and $s = L$ |
| IC | Initial conditions | Straight rod at rest: $\mathbf{x}(s, 0) = \mathbf{x}_0 + s\hat{\mathbf{e}}_x,\ \mathbf{v} = \boldsymbol{\kappa} = \boldsymbol{\omega} = \mathbf{0}$ |
| $\boldsymbol{\theta}$ | Parameters | $E$=$10^5$ Pa, $\nu$=0.5, $\rho$=1200 kg/m$^3$, $R$=0.02 m, $c_t$=0.01, $c_n$=0.05, $\gamma$=0.002 |
| Control | CPG forcing | $\mathbf{a} = (A, f, k, \phi_0, b) \in \mathbb{R}^5$ enters as time-varying rest curvature $\boldsymbol{\kappa}_0(s, t)$ |

The governing PDEs are:

$$\rho A \frac{\partial \mathbf{v}}{\partial t} = \frac{\partial \mathbf{F}}{\partial s} + \mathbf{f}_{\text{ext}} \qquad \text{(linear momentum)} \tag{1a}$$

$$\rho \mathbf{I} \frac{\partial \boldsymbol{\omega}}{\partial t} = \frac{\partial \mathbf{M}}{\partial s} + \mathbf{m}_{\text{ext}} + \frac{\partial \mathbf{x}}{\partial s} \times \mathbf{F} \qquad \text{(angular momentum)} \tag{1b}$$

with constitutive closure:

$$\mathbf{F} = \mathbf{S}\,(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_0), \quad \mathbf{M} = \mathbf{B}\,(\boldsymbol{\kappa} - \boldsymbol{\kappa}_0), \quad \mathbf{S} = \mathrm{diag}(GA, GA, EA), \quad \mathbf{B} = \mathrm{diag}(EI, EI, GJ) \tag{2}$$

Spatial discretization on a staggered grid ($N$=21 nodes, $N_e$=20 elements) reduces the PDE to 124 coupled first-order ODEs:

$$\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s},\, \mathbf{a},\, t), \qquad \mathbf{s} \in \mathbb{R}^{124}, \ \ \mathbf{a} \in \mathbb{R}^5 \tag{3}$$

Position Verlet ($\Delta t = 0.001\,\mathrm{s}$, $n_{\text{sub}} = 500$ substeps per RL step of $\Delta t_{\text{ctrl}} = 0.5\,\mathrm{s}$) produces the transition operator:

$$\mathbf{s}_{t+1} = T(\mathbf{s}_t,\, \mathbf{a}_t) = \underbrace{V_{\Delta t}(\,\cdot\,;\, \boldsymbol{\kappa}^{\text{rest},499}) \circ \cdots \circ V_{\Delta t}(\,\cdot\,;\, \boldsymbol{\kappa}^{\text{rest},0})}_{\text{500 substeps, each with distinct rest curvature}}(\mathbf{s}_t) \tag{4}$$

See Appendix A for detailed PDE derivations, staggered grid operators, external force handling, and the DisMech formulation.

Table 4: State vector $\mathbf{s}_t \in \mathbb{R}^{124}$ layout (Elastica).

| Slice | Variable | Count | Meaning | Equation |
|---|---|---|---|---|
| `[0:21]` | $x_i$ | 21 | Node $x$-positions (m) | Position kinematics |
| `[21:42]` | $y_i$ | 21 | Node $y$-positions (m) | Position kinematics |
| `[42:63]` | $\dot{x}_i$ | 21 | Node $x$-velocities ($\mathrm{m\,s^{-1}}$) | Linear momentum ODE |
| `[63:84]` | $\dot{y}_i$ | 21 | Node $y$-velocities ($\mathrm{m\,s^{-1}}$) | Linear momentum ODE |
| `[84:104]` | $\psi_e$ | 20 | Element yaw angle (rad) | Orientation kinematics |
| `[104:124]` | $\omega_{z,e}$ | 20 | Element angular velocity ($\mathrm{rad\,s^{-1}}$) | Angular momentum ODE |

Algorithm 1 expands the full Elastica transition operator.

**Algorithm 1** Elastica Transition Operator $T(\mathbf{s}_t,\, \mathbf{a}_t,\, t_{\text{step}}) \to \mathbf{s}_{t+1}$

**Require:** State $\mathbf{s}_t \in \mathbb{R}^{124}$, action $\mathbf{a}_t \in \mathbb{R}^5$, step time $t_{\text{step}}$
**Ensure:** Next state $\mathbf{s}_{t+1} \in \mathbb{R}^{124}$
$\mathbf{s} \leftarrow \mathbf{s}_t$
**for** $n = 0, 1, \ldots, 499$ **do**
  $t^n \leftarrow t_{\text{step}} + n \cdot \Delta t$
  Update rest curvature: $\kappa_v^{\text{rest}} \leftarrow A \sin(2\pi k \cdot s_v + 2\pi f \cdot t^n + \phi_0) + b$ for $v = 1, \ldots, 19$
  Verlet half-step: $\mathbf{q}^{1/2} \leftarrow \mathbf{q} + \frac{1}{2}\Delta t\, \dot{\mathbf{q}}$
  Compute geometry: edge vectors, tangents, strain, curvature
  Compute internal forces (shear–stretch) and torques (bending)
  Compute external forces: RFT friction + damping + gravity
  Compute accelerations: $\ddot{\mathbf{x}} \leftarrow (\mathbf{F}^{\text{int}} + \mathbf{F}^{\text{ext}})/\mathbf{m}$
  Verlet velocity update + full-step position update
**end for**
**return** $\mathbf{s}_{t+1} = (\mathbf{q},\, \dot{\mathbf{q}})$

*Cost:* $\sim$377,000 FLOPs/step ($\sim$46 ms on CPU). The CPG phase advances by $\Delta\phi = 2\pi f \Delta t$ per substep — the surrogate must learn the *integrated effect* of the traveling wave across 500 distinct curvature configurations.

### 3.2 CPG Control

The Central Pattern Generator drives locomotion by imposing time-varying rest curvature on the rod. When the rest curvature differs from the rod's current curvature, elastic bending moments drive the rod toward the target shape. As the CPG sweeps a sinusoidal curvature wave along the body, anisotropic ground friction (higher normal than tangential) converts the lateral undulation into net forward thrust—the same mechanism underlying biological serpentine locomotion. The RL agent selects five CPG wave parameters per control step:

Table 5: Action vector $\mathbf{a}_t \in \mathbb{R}^5$: CPG wave parameters.

| Symbol | Name | Range | Units |
|---|---|---|---|
| $A$ | Amplitude | $[0, 5]$ | $\mathrm{rad\,m^{-1}}$ |
| $f$ | Frequency | $[0.5, 3.0]$ | Hz |
| $k$ | Wave number | $[0.5, 3.5]$ | wavelengths/body |
| $\phi_0$ | Phase offset | $[0, 2\pi]$ | rad |
| $b$ | Turn bias | $[-2, 2]$ | $\mathrm{rad\,m^{-1}}$ |

The serpenoid rest curvature at each Voronoi node:

$$\kappa_v^{\text{rest}}(t) = A \sin(2\pi k \cdot s_v + 2\pi f \cdot t + \phi_0) + b, \qquad v = 1, \ldots, 19. \tag{5}$$

### 3.3 Physics Backends

This work uses two Cosserat rod simulators with fundamentally different numerical strategies. Table 6 summarizes the key differences.

Table 6: Comprehensive comparison of PyElastica and DisMech.

| **Property** | **PyElastica** | **DisMech** |
|---|---|---|
| *Solver* | | |
| Physics model | Cosserat rod theory | Discrete Elastic Rods (DER) |
| Integration scheme | Explicit symplectic (Verlet) | Implicit backward Euler (Newton) |
| Timestep per call | $\Delta t_{\text{sub}} = 0.001\,\text{s}$ | $\Delta t = 0.05\,\text{s}$ |
| Steps per RL action | 500 substeps | 1 step |
| Per-step cost | $\sim$46 ms (500 explicit evals) | $\sim$12 ms (1 Newton solve) |
| *Supported forces* | | |
| Shear–stretch / Bending / Torsion | ✓ | ✓ |
| RFT (anisotropic drag) | ✓ | — |
| Self-contact | — | ✓(FCL) |
| *Control* | | |
| Curvature mechanism | Elastic rest shape ($\kappa_0$) | Hard constraint (natural strain) |
| CPG resolution | 500 curvature updates/step | 1 curvature snapshot/step |
| *Metadata* | | |
| Theory | Cosserat rod | 2008 DER [21]; 2010 [22] |
| Simulator | 2021 PyElastica [2] | 2024 DisMech [23] |
| Python framework | (same) | 2025 Py-DiSMech [24] |
| Repository | `github.com/GazzolaLab/PyElastica` | `github.com/StructuresComp/dismech-rods` |
| Language | Python (Numba JIT) | C++ (Python bindings) |

The fundamental trade-off: PyElastica's 500 substeps capture continuous CPG evolution but cost $4\times$ more wall-clock time. DisMech's single implicit step is faster and unconditionally stable, but collapses intra-step dynamics. For surrogate modeling, Elastica presents a harder approximation target.

The DisMech ecosystem has evolved significantly beyond the original DER formulation. Choi et al. [23] introduced DisMech as a full simulator with implicit contact handling and real2sim validation, achieving an order-of-magnitude speedup over prior DER implementations. Lahoti et al. [24] subsequently released Py-DiSMech, a vectorized Python reimplementation that further improves computational efficiency while adding PI-based shape control and modular actuation interfaces.

# 4 Neural Surrogate Models

Reinforcement learning for soft robots requires millions of environment interactions, yet first-principles Cosserat rod simulators are expensive (Section 3.3). Neural surrogate models address this bottleneck by replacing the physics solver with a learned transition function $f_\theta$ that approximates the state-to-state mapping in a single GPU forward pass ($\sim$0.001 ms), targeting four-to-five orders-of-magnitude speedup. This section presents the implemented data-driven surrogate and outlines two physics-informed extensions — a physics regularizer and a domain-decoupled PINN — as proposed future directions. All surrogate experiments ran on the hardware in Table 7.

Table 7: Hardware for data collection and surrogate training.

| **Component** | **Specification** |
|---|---|
| GPU | $1\times$ Tesla V100-PCIE-16 GB (Volta, 5120 CUDA cores) |
| CPU | 24 cores / 48 threads |
| RAM | 270 GB |
| CUDA | 12.8 |
| Python | CPython 3.12 |

## 4.1 Data Collection

Training data is generated by 16 independent PyElastica worker processes:

1. Actions sampled from 5D CPG space using Sobol quasi-random sequences (50% Sobol, 50% uniform).
2. 30% of episodes perturbed: random position offsets, angular velocity noise ($\sigma_\omega = 1.5\,\text{rad/s}$).
3. Each worker stores transitions $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}, t_{\text{start}})$ to batch files.
4. Health monitor respawns stalled workers.

Table 8: Data collection pipeline parameters.

| **Parameter** | **Value** |
|---|---|
| Parallel workers | 16 |
| Transition size (with forces) | $\sim$2055 bytes |
| Target dataset size | $\sim$10 GB ($\sim$5M transitions) |
| Per-worker throughput | $\sim$22 transitions/s |
| Aggregate throughput | $\sim$350 transitions/s |

**Dataset quality.** The final dataset contains 4,336,600 single-step transitions. Table 9 summarizes automated validation.

Table 9: Data quality validation results.

| **Metric** | **Value** | **Threshold** | **Status** |
|---|---|---|---|
| NaN/Inf rate | 0.00% | $< 0.1\%$ | Pass |
| Duplicate transitions | 3.84% | $< 1\%$ | Fail |
| Constant features | 0 | 0 | Pass |
| Outlier rate ($> 5\sigma$) | 0.096% | $< 0.5\%$ | Pass |
| Action 5D fill fraction | 74.5% | $> 5\%$ | Pass |
| Per-dim action fill | 100% | $> 90\%$ | Pass |

The 3.84% duplicate rate arises from identical initial conditions under 1-step episodes and amounts to implicit oversampling — retained without deduplication (4,170,082 unique transitions).

## 4.2 MLP Surrogate

### 4.2.1 Input Representation

The surrogate input concatenates three components:

$$\mathbf{z}_t = \left[\bar{\mathbf{s}}_t \parallel \mathbf{a}_t \parallel \boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_{N_e}\right] \in \mathbb{R}^{189} \tag{6}$$

Table 10: Surrogate input decomposition ($189 = 124 + 5 + 60$).

| Component | Dim | Description |
|---|---|---|
| $\bar{\mathbf{s}}_t$ | 124 | Z-score normalized rod state |
| $\mathbf{a}_t$ | 5 | Normalized CPG action $\in [-1, 1]^5$ |
| $\boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_{20}$ | 60 | Per-element phase triplets |

#### 4.2.2 Per-Element CPG Phase Encoding

For element $e$, the phase triplet is:

$$\boldsymbol{\phi}_e = \big(\sin(2\pi f \cdot t_e),\ \cos(2\pi f \cdot t_e),\ \kappa_e^{\text{actual}}\big), \qquad t_e = \frac{k \cdot s_e}{f} \tag{7}$$

This provides the exact phase of the serpenoid wave at each element's location, plus the instantaneous curvature $\kappa_e^{\text{actual}}$ for the control error. Phase features are computed on-the-fly from stored action and $t_{\text{start}}$.

An earlier V1 global phase encoding ($\mathbb{R}^2$) achieved only $R^2 = 0.23$ for angular velocity; the V2 per-element encoding significantly improved prediction.

#### 4.2.3 Architecture and Prediction

The surrogate predicts a normalized state delta:

$$f_\theta(\mathbf{z}_t) \approx T(\mathbf{s}_t, \mathbf{a}_t) - \mathbf{s}_t, \qquad \hat{\mathbf{s}}_{t+1} = \mathbf{s}_t + \boldsymbol{\sigma}_\Delta \odot f_\theta(\mathbf{z}_t) + \boldsymbol{\mu}_\Delta \tag{8}$$

Network architecture:

$$\mathbb{R}^{189} \xrightarrow{\text{Linear}} \big[\text{Linear}(512) \to \text{LayerNorm} \to \text{SiLU}\big]^{\times 3} \xrightarrow{\text{Linear}} \mathbb{R}^{124} \tag{9}$$

- Output layer initialized to zero (default prediction: mean empirical transition)
- Total trainable parameters: $\sim$659K
- Delta prediction concentrates the target distribution and provides an implicit residual connection

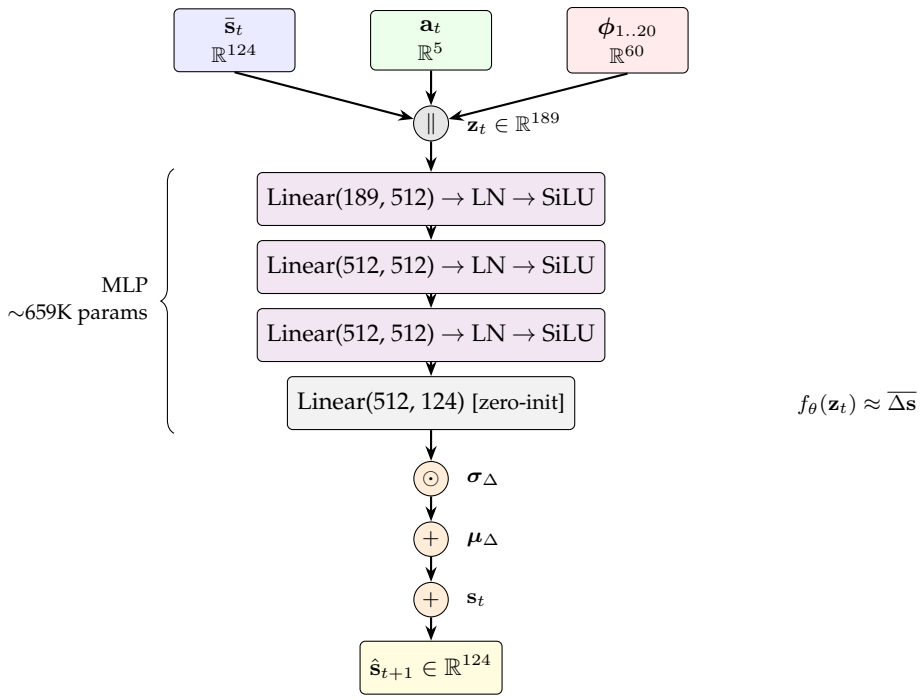



Figure 1: Surrogate architecture. Three input streams are concatenated into $\mathbf{z}_t \in \mathbb{R}^{189}$, passed through three LayerNorm–SiLU blocks, and projected to a 124-dim normalized delta. An affine reconstruction recovers the next state $\hat{\mathbf{s}}_{t+1}$.

#### 4.2.4 Training

The surrogate is trained with a single-step MSE loss on normalized state deltas:

$$\mathcal{L}_{\text{single}} = \frac{1}{N} \sum_{i=1}^{N} \left\| f_\theta(\mathbf{z}_t^{(i)}) - \overline{\Delta \mathbf{s}}_{\text{true}}^{(i)} \right\|^2 \tag{10}$$

**Density-weighted sampling.** Each training sample is weighted by inverse estimated local density:

$$w^{(i)} = \frac{1}{\hat{p}(\mathbf{c}^{(i)})}, \qquad \mathbf{c}^{(i)} = \left[\bar{x}_{\text{CoM}},\ \bar{y}_{\text{CoM}},\ \|\dot{\mathbf{x}}\|,\ |\bar{\omega}_z|\right] \tag{11}$$

Weights are clipped at $10\times$ the mean to prevent rare samples from dominating.

#### 4.2.5 Hyperparameters

Table 11: Surrogate training hyperparameters.

| **Hyperparameter** | **Value** |
|---|---|
| Architecture | 4-layer MLP ($4 \times 1024$) |
| Activation / Normalization | SiLU / LayerNorm (pre-activation) |
| Optimizer | Adam ($\beta_1$=0.9, $\beta_2$=0.999) |
| Learning rate | $1 \times 10^{-4}$ (cosine, 5-epoch warmup) |
| Batch size | 4096 (auto-scaled to $\sim$245K effective) |
| Mixed precision | bfloat16 |
| Early stopping | Patience 30 (validation MSE) |
| Total parameters | $\sim$3.43M |

Table 12: Per-run configuration differences and results.

| | **Run A** | **Run B (best)** | **Run C** |
|---|---|---|---|
| State dim | 128 | 128 | 130 |
| Dropout | 0.0 | 0.1 | 0.1 |
| Weight decay | $10^{-5}$ | $10^{-4}$ | $10^{-4}$ |
| Epochs trained | 80 | 110 | 57 |
| Best val. loss | 0.712 | **0.601** | 0.733 |
| Overall $R^2$ | — | **0.393** | 0.258 |

### 4.3 MLP Results

The best surrogate (Run B) reached a validation loss of 0.601 and an overall $R^2$ of 0.393 after 110 epochs on 3.9M transitions.

Table 13: Surrogate single-step prediction accuracy by state component (Run B, epoch 110).

| State component | $R^2$ | MSE |
|---|---|---|
| CoM position ($\bar{x}, \bar{y}$) | 0.852 | $2.81 \times 10^{-7}$ |
| Relative positions ($\Delta x_i, \Delta y_i$) | 0.773 | $\sim 3.2 \times 10^{-5}$ |
| Angular velocity ($\omega_{z,e}$) | 0.320 | 1.442 |
| Heading ($\theta$) | 0.287 | $2.49 \times 10^{-3}$ |
| Yaw ($\psi_e$) | 0.173 | $4.78 \times 10^{-2}$ |
| Velocities ($\dot{x}_i, \dot{y}_i$) | 0.145 | $8.10 \times 10^{-2}$ / $2.13 \times 10^{-1}$ |
| Overall | **0.393** | 0.601 |

Geometric quantities (CoM, relative positions) converge within 20 epochs. Velocity and angular components improve gradually and have not plateaued by epoch 110, suggesting longer training or architectural changes could help.

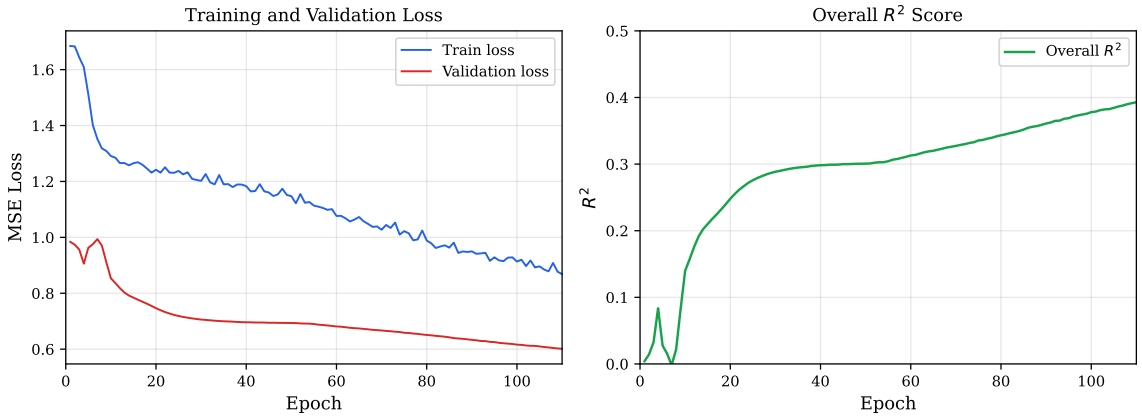



Figure 2: Training and validation MSE loss (left) and overall $R^2$ (right) over 110 epochs for Run B. The persistent train–validation gap indicates mild overfitting despite dropout and weight decay.

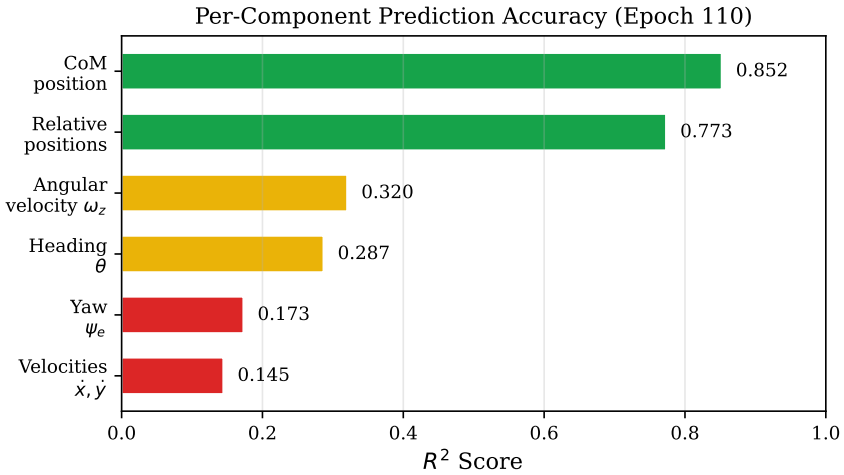



Figure 3: Per-component $R^2$ at epoch 110 (Run B). Geometric quantities are well predicted; velocity components remain challenging.

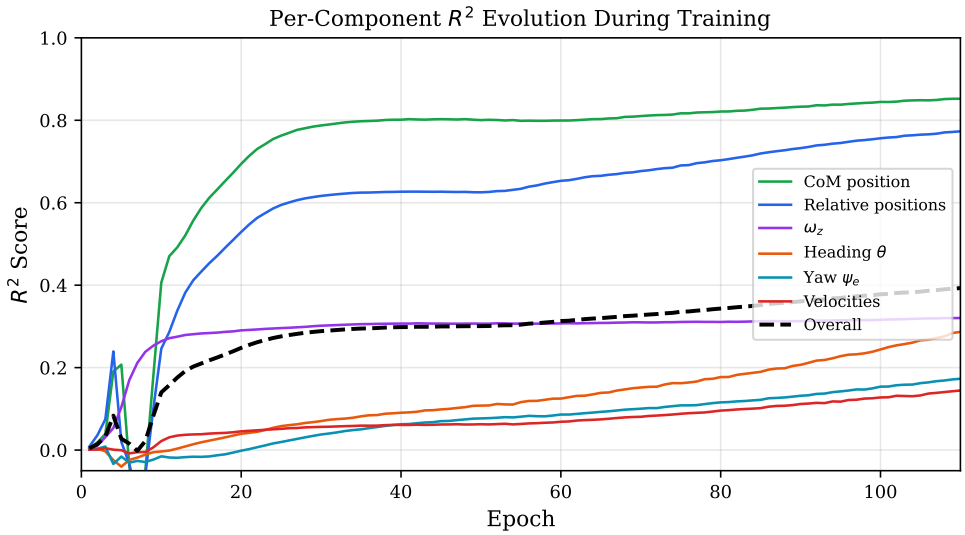



Figure 4: Per-component $R^2$ evolution during training. Position components converge within 20 epochs; angular and velocity components improve gradually.

Table 14 summarizes the four main challenges identified during surrogate development.

Table 14: Surrogate prediction challenges, root causes, and mitigations.

| **Challenge** | **Root cause** | **Mitigation** |
|---|---|---|
| Weak $\omega_z$ prediction ($R^2 = 0.320$) | Control step (0.5 s) spans ∼1.5 CPG cycles; $\omega_z$ reverses sign multiple times, creating a many-to-one mapping | Per-element phase encoding (Section 4.2.2) provides cycle position; fundamental aliasing remains |
| Training data bias | Policy rollouts cluster around learned behaviors; rod starts from straight configuration, leaving bent states underrepresented | Sobol sampling (50%), state perturbation (30%), curvature noise; full 189-dim coverage remains intractable |
| Curvature granularity | Curvature initially applied once per 0.5 s control step, creating discontinuous forcing over 500 substeps | Moved update into substep loop → near-continuous traveling wave |
| Persistent overfitting (Figure 2) | 4.3M transitions do not fully cover state-action manifold; model memorizes trajectories | Dropout ($p$=0.1), weight decay ($\lambda$=$10^{-4}$); gap persists—more diverse data or attention architectures may help |

### 4.4 Physics-Informed Extensions (Proposed)

Two physics-informed surrogate extensions are proposed as future work but have not been implemented: a *physics regularizer* that adds soft algebraic constraints (kinematics, curvature–moment consistency, bounded energy) to the data-driven loss, and a *domain-decoupled PINN* (DD-PINN) [25, 26] that removes time as a network input, instead outputting parameters of an analytically differentiable ansatz — yielding exact initial conditions, closed-form gradients (no autodiff), and $5$–$38\times$ faster training than standard PINNs. Full formulations, loss functions, architectural details, and a four-way comparison (PDE solver vs. data-driven vs. PINN vs. DD-PINN) are given in Appendix B.

### 4.5 Surrogate Comparison

The data-driven surrogate achieves highest throughput and simplest training but provides no physics guarantees. The physics-regularized variant would add soft constraints at minimal overhead ($\sim 0.1$ ms GPU). The DD-PINN would satisfy initial conditions exactly, eliminate autodiff cost, and enforce governing equations — at the cost of ansatz design. A detailed four-way comparison is provided in Appendix B. Implementing and benchmarking these two physics-informed extensions is a priority for future work.

### 4.6 Surrogate Validation: RL on Elastica (E1)

Before training RL on the surrogate, we establish a direct-simulation baseline on the Elastica backend. This experiment also characterizes the sim-to-sim gap between the two physics backends used in this work.

#### 4.6.1 Environment and Training Setup

Direct RL training uses the Elastica physics simulator as the environment, running on $1\times$ Tesla V100-PCIE-16 GB (24 cores, 270 GB RAM, CUDA 12.8). At each step, the agent selects CPG parameters $\mathbf{a}_t \in [-1, 1]^5$, the simulator advances via 500 substeps, and a displacement-based reward is computed:

$$r_t = x_{\text{CoM}}(t+1) - x_{\text{CoM}}(t) \tag{12}$$

PPO is used as the policy optimization algorithm with TorchRL.

Table 15: PPO training hyperparameters (Session 10, best).

| **Hyperparameter** | **Value** |
|---|---|
| Parallel environments | 16 |
| Total frames | $2 \times 10^6$ |
| Frames per batch | 8192 |
| Mini-batch size | 256 |
| PPO clip $\epsilon$ / PPO epochs | 0.2 / 4 |
| Learning rate | $1 \times 10^{-4}$ (linear decay to $10^{-5}$) |
| Discount $\gamma$ / GAE $\lambda$ | 0.99 / 0.95 |
| Entropy coefficient / Value coefficient | 0.02 / 0.5 |
| Max gradient norm / Target KL | 0.5 / 0.01 |
| Max episode steps | 500 |

### 4.6.2 Direct Simulation Results

Training exhibits three phases: (1) *learning* (0–164k steps, peak rolling average 140.0), (2) *decline* (164k–574k, policy overshooting in CPG space), (3) *steady state* (574k–2M, stabilized $\sim$102). Best checkpoint reward: 156.66 at step 155,648.

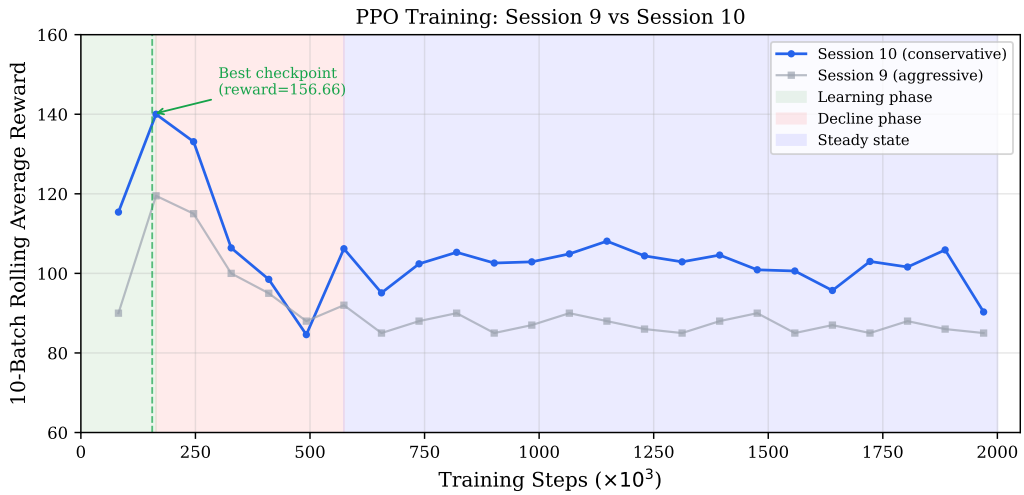



Figure 5: PPO training curves. Session 10 (conservative hyperparameters) achieves higher peak reward and more stable behavior than Session 9 (aggressive hyperparameters).

**Evaluation.** 20 deterministic-action episodes with randomized initial headings:

Table 16: Direct Elastica RL training and evaluation summary.

| **Metric** | **Value** |
|---|---:|
| Training time | $\sim$9 h |
| Throughput | $\sim$55–60 FPS |
| Peak rolling reward | 140.0 |
| Best checkpoint reward | 156.66 |
| Eval mean displacement | $1.495 \pm 0.13$ m |
| Eval mean reward | $64.90 \pm 1.44$ |
| Goal success rate | 0/20 (0%) |

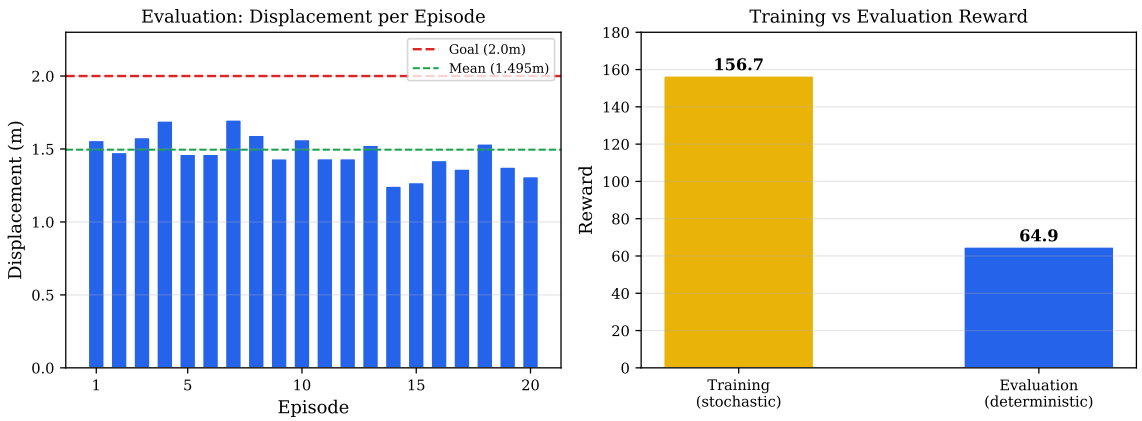



Figure 6: Evaluation of best Session 10 checkpoint. Left: displacement per episode. Right: training vs. evaluation reward gap.

The $2.4\times$ gap between training (156.66) and evaluation (64.90) rewards is attributed to stochastic actions during training, favorable heading distributions, and goal-reaching bonuses.

#### 4.6.3 Surrogate Environment Results

Surrogate-based RL training has not yet been conducted. The moderate overall $R^2 = 0.393$ and poor velocity prediction ($R^2 = 0.145$) raise concerns about multi-step rollout fidelity. The key risk is that the policy may exploit surrogate artifacts rather than learning physically grounded gaits that transfer to the Elastica simulator.

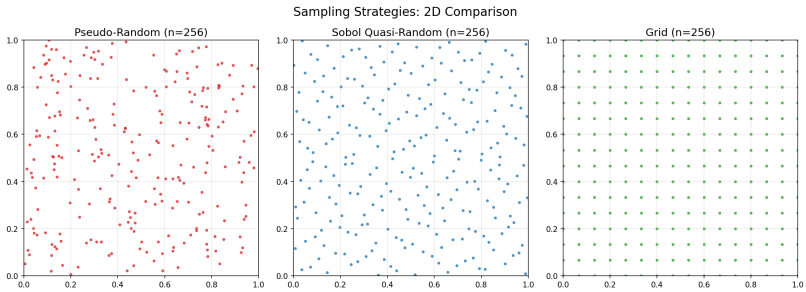



Figure 7: Action-space sampling comparison: Sobol quasi-random vs. pseudo-random coverage for surrogate data collection.

#### 4.6.4 Challenges

- **Computational cost:** $\sim$46 ms/step $\times$ 16 workers $=$ $\sim$350 steps/s; $2 \times 10^6$ steps requires $\sim$36 h. DisMech is $\sim$4$\times$ faster but still CPU-bound.
- **Exploration in CPG space:** Large regions produce no locomotion (zero amplitude, extreme frequencies). Curriculum learning over parameter subsets improves efficiency.
- **Reward sparsity:** Distance-based rewards require discovering forward-moving gaits before receiving signal.
- **Backend discrepancy:** DisMech-trained policies do not transfer to Elastica due to friction, curvature mechanism, and temporal resolution differences.

## 5 Reward Design for Soft Manipulator Control

This section presents six experiment groups (E2–E7) totaling $\sim$28 individual runs on the DisMech backend. Table 17 indexes all experiments with a unified identifier; E1 (Elastica) is documented in Section 4.6.

Table 17: Experiment registry. E1 uses Elastica (Section 4.6); all other experiments use DisMech PPO on the `follow_target` task (Section 5.1.1).

| ID | Description | Envs | Frames | Key finding |
|---|---|---|---|---|
| E1 | Elastica PPO locomotion (§4.6, Sec. 4) | 16 | 2M | 1.5 m displacement; 0% goal success |
| E2 | Reward-shaping ablation (§5.2) | 100 | 5M$\times$5 | Original reward best (+52% vs. baseline) |
| E3 | Reward landscape diagnostics (§5.3) | — | random | 70% of steps yield $r < 0.05$ |
| E4 | Extended training (§5.4) | 100 | 50M | Plateau at $d \approx 0.6$; not a budget issue |
| E5 | SAC vs. PPO stability (§5.5) | 100 | 20M/50M | SAC gradients explode past $\sim$3M frames |
| E6 | Validation ablation, A1–C5 (§5.7) | 32 | 3.6M$\times$12 | Curriculum dominant; best $d$=0.543 |
| E7 | Steepness and env scaling (§5.8) | 100 | 10.5M$\times$4 | $k$=5 wins; 100 envs 24% worse than 32 |

## 5.1 DisMech: Task and Setup

This subsection presents PPO experiments on the DisMech physics backend for the `follow_target` task from Choi and Tong [16]. While the original paper uses a Gymnasium-based environment with a custom training framework, our implementation wraps DisMech in TorchRL's `EnvBase` and trains via `SyncDataCollector` with `ParallelEnv`.

All experiments ran on 2× RTX A4000 16 GB GPUs, 16 cores / 32 threads, 270 GB RAM, CUDA 12.8.

### 5.1.1 Task Description

The `follow_target` task requires a clamped soft manipulator (Cosserat rod, 21 nodes, 1 m length) to track a moving target in 3D. The target moves at 0.05 m/s within a workspace of radius 0.3–0.9 m. At each control step, the agent selects 5 curvature increments $\Delta\kappa \in [-1, 1]$, applied as natural strain through DisMech's bend springs.

Table 18: Environment configuration (common to all runs).

| **Parameter** | **Value** |
|---|---|
| Physics backend | DisMech (C++ implicit Euler) |
| Rod discretization | 21 nodes, 20 elements |
| Observation dim | 34 (positions, curvatures, target) |
| Action dim | 5 ($\Delta\kappa$ curvature increments) |
| Max episode steps | 200 |
| Control timestep | 0.05 s (2 substeps at dt=0.05 s) |
| Target speed | 0.05 m/s |
| Workspace radius | 0.3–0.9 m |

### 5.1.2 Training Configuration

All five initial ablation runs use identical PPO hyperparameters, differing only in the reward function. Table 19 shows the configuration of each run.

Table 19: Run configurations for the reward-shaping ablation study.

| | **Original** | **Baseline** | **Heading** | **PBRS+Base** | **PBRS-Only** |
|---|---|---|---|---|---|
| Algorithm | PPO | PPO | PPO | PPO | PPO |
| Network | 4×512 ReLU | 4×512 ReLU | 4×512 ReLU | 4×512 ReLU | 4×512 ReLU |
| Learning rate | $10^{-4}$ | $10^{-4}$ | $10^{-4}$ | $10^{-4}$ | $10^{-4}$ |
| Clip $\epsilon$ | 0.2 | 0.2 | 0.2 | 0.2 | 0.2 |
| Epochs per batch | 10 | 10 | 10 | 10 | 10 |
| Mini-batch size | 1024 | 1024 | 1024 | 1024 | 1024 |
| Frames per batch | 8192 | 8192 | 8192 | 8192 | 8192 |
| Entropy coeff | 0.1 | 0.1 | 0.1 | 0.1 | 0.1 |
| Parallel envs | 100 | 100 | 100 | 100 | 100 |
| Total frames | 5M | 5M | 5M | 5M | 5M |
| Seed | 42 | 42 | 42 | 42 | 42 |
| Curriculum | Yes | Yes | Yes | Yes | Yes |
| *Reward-specific parameters* | | | | | |
| Base reward $r_{\text{dist}}$ | Yes | Yes | Yes | Yes | No |
| Improvement $w_{\text{imp}}$ | 10.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| Heading weight $w$ | 0.0 | 0.0 | 0.3 | 0.0 | 0.0 |
| PBRS $\gamma$ | 0.0 | 0.0 | 0.0 | 0.99 | 0.99 |

**Replication adjustments.** Two configuration errors in our initial DisMech replication were identified by comparison with the paper's Table A.3. The rod radius was set to $r = 0.001\,\text{m}$ (50× too thin) and Young's modulus to $E = 2\,\text{MPa}$ (5× too soft), producing a near-limp structure that capped reward at ∼31. Correcting these to $r = 0.05\,\text{m}$ and $E = 10\,\text{MPa}$ matched the paper's physical setup. Additionally, the raw 34-dimensional observations have heterogeneous scales (positions in $[0, 1]\,\text{m}$, curvatures varying by orders of magnitude). We apply running-mean observation normalization (`ObservationNorm` with 200-iteration initialization) so that all input dimensions have approximately unit variance, which stabilized value function learning (explained variance improved from oscillating around $0$ to a steady ∼0.65).

## 5.2 Reward Variants and Initial Ablation (E2)

### 5.2.1 Reward Variants

Choi and Tong [16] define the follow-target reward with two terms:

$$r(s, s') = \underbrace{\exp\big(-5\, d(s')\big)}_{r_{\text{dist}}} + \underbrace{10\,\big(d(s) - d(s')\big)}_{r_{\text{improve}}}, \qquad d(s) = \|\mathbf{x}_{\text{tip}} - \mathbf{x}_{\text{target}}\|. \tag{13}$$

The first term is a bounded distance reward in $[0, 1]$, approaching 1 as the tip reaches the target. The second is an improvement bonus proportional to the step-wise distance reduction. The scaling factor of 5 was chosen to match the workspace radius; the improvement multiplier of 10 amplifies the approach signal.

**Simplification for PPO.** The original reward was designed for SAC, which averages over a replay buffer and tolerates high reward variance. For on-policy PPO, the improvement bonus with its $10\times$ multiplier creates a reward SNR of only 0.08 (the signal is 8% of the noise), causing the value function to fail. We therefore use only the distance component as the base reward:

$$r_{\text{dist}}(s) = \exp\big(-5\, d(s)\big). \tag{14}$$

This raised the SNR from 0.08 to 1.02 (a $13\times$ improvement) and stabilized explained variance from 0.24 to 0.65. We then investigate three augmentations to this simplified base reward, described below.

**Heading reward.** A cosine-similarity bonus for pointing the tip toward the target:

$$r_{\text{heading}}(s) = \frac{1 + \cos\theta}{2}, \qquad r_{\text{total}} = (1 - w)\, r_{\text{dist}} + w\, r_{\text{heading}}, \tag{15}$$

where $\theta$ is the angle between the tip tangent and the tip-to-target direction, and $w \in [0, 1]$ is a mixing weight. Both components lie in $[0, 1]$, so the total reward is bounded.

**Potential-Based Reward Shaping (PBRS).** Following Ng et al. [27] (with the equivalence to Q-value initialization established by Wiewiora [28]), we define a potential $\Phi(s) = -d(s)$ and add the shaping reward:

$$F(s, s') = \gamma_{\text{pbrs}}\, \Phi(s') - \Phi(s) = d(s) - \gamma_{\text{pbrs}}\, d(s'), \tag{16}$$

where $\gamma_{\text{pbrs}}$ is a discount factor (we use 0.99). This is guaranteed not to change the optimal policy. When combined with the base reward: $r_{\text{total}} = r_{\text{dist}} + F(s, s')$.

**PBRS-only.** An ablation where the base reward is removed entirely, using only the shaping signal: $r_{\text{total}} = F(s, s')$.

**Curriculum learning.** All runs use curriculum learning: target speed ramps from 20% to 100% over the first 100 episodes per worker, avoiding reward-scale shock at initialization.

**Fair comparison metric.** Different reward functions produce incomparable total rewards (e.g., heading inflates the total by up to 72%). We log per-component rewards and use $r_{\text{dist}} = \exp(-5d)$ as the *fair metric* for cross-run comparison, since it depends only on the physical outcome (distance to target) and is independent of the reward function.

### 5.2.2 Results

Table 20 summarizes the training outcomes. All metrics in the "fair metric" column use $r_{\text{dist}} = \exp(-5d)$ averaged over the final 20% of training, enabling direct comparison regardless of the reward function. The signal-to-noise ratio (SNR) is computed as $|\bar{r}|/\sigma_r$ over all training batches.

Table 20: Training results on follow_target (5M frames each). Fair metric is $r_{\text{dist}} = \exp(-5d)$ averaged over the final 20% of episode entries.

| Run | $r_{\text{dist}}$ (fair) | vs. Baseline | SNR | Episode Reward | Best Episode |
|---|---|---|---|---|---|
| Original ($w_{\text{imp}}$=10) | **0.143** | +52.1% | 1.72 | 28.2 | 90.2 |
| PBRS+Base | 0.109 | +16.9% | 1.00 | 27.4 | 77.9 |
| Baseline | 0.094 | — | 1.20 | 18.7 | 61.3 |
| Heading ($w$=0.3) | 0.072 | −23.3% | 1.82 | 39.1 | 63.0 |
| PBRS-Only | 0.067 | −28.7% | 0.10 | 1.8 | 2.9 |

Key observations:

- **The original two-term reward is the best performer** at +52.1% over baseline on the fair metric, with the highest best-episode reward (90.2). Despite our earlier SNR analysis suggesting the $10\times$ improvement bonus would destabilize PPO, curriculum learning appears to mitigate this: during the warmup phase the slow-moving target produces small, consistent distance improvements, keeping the bonus well-scaled until the policy matures.
- **PBRS+Base is the second best** at +16.9% over baseline. The shaping signal $F = d(s) - 0.99\, d(s')$ provides a per-step gradient toward the target, accelerating learning without distorting the optimal policy.
- **Heading reward hurts by 23.3%.** Although the heading run achieves inflated episode rewards (39.1 vs. 18.7), this reflects the heading component contributing ∼72% of the total reward. On the fair metric, heading reduces distance-tracking performance—a case of reward hacking where the agent optimizes alignment at the expense of proximity.
- **PBRS-only is insufficient** (−28.7%). Removing the base reward leaves only the shaping signal, which has critically low SNR (0.10, vs. the 0.5 threshold for reliable learning).
- **Curriculum rescues high-variance rewards.** The original reward's SNR without curriculum was 0.08 (Section 5.2.1); with curriculum it reaches 1.72. The slow ramp prevents the large improvement bonus from overwhelming the value function during early training.

**Reward curves.** Figure 8 shows the fair metric and distance to target over training. The original reward (purple) separates from all other runs after ∼1.5M frames, reaching the highest fair metric. PBRS+Base (green) is the second-best, separating after ∼2M frames. The baseline (blue) and heading (orange) both improve slowly; PBRS-Only (red, dashed) remains flat near the bottom.

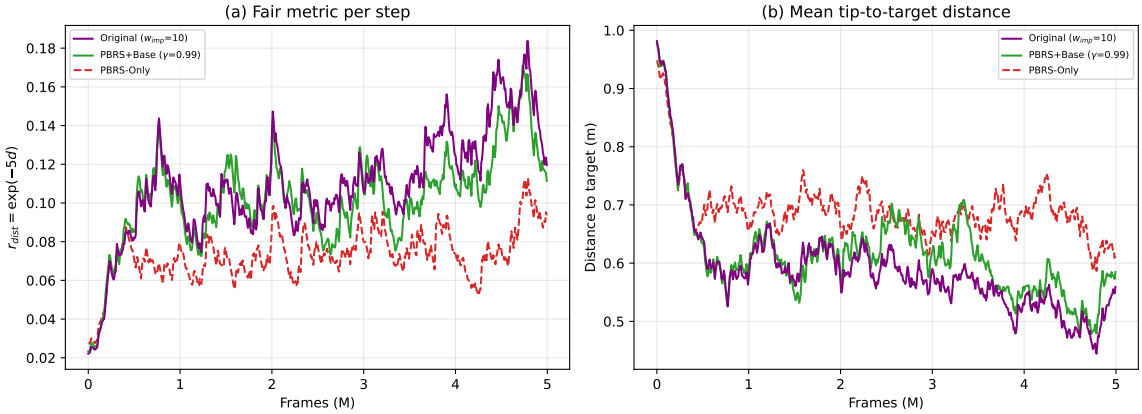



Figure 8: PPO reward-shaping ablation on follow_target (5M frames, 100 envs, seed 42). (a) Fair metric $r_{\text{dist}} = \exp(-5d)$ per step, EMA-smoothed ($\alpha = 2/41$). (b) Mean tip-to-target distance. The original two-term reward (purple) outperforms all variants; PBRS+Base (green) is second; heading (orange) and PBRS-Only (red) underperform baseline.

### 5.2.3 Discussion

Table 21 summarizes the key findings from the ablation.

Table 21: Key findings from the reward-shaping ablation (5M frames, 100 envs).

| Finding | Evidence | Implication |
|---|---|---|
| Reward design > hyperparameter tuning | 52.1% improvement range across reward variants with identical hyperparameters | Reward function is the primary lever for this task |
| Curriculum × reward interaction | Original reward SNR: 0.08 without curriculum, 1.72 with—same function, opposite outcomes | Reward and curriculum are coupled design choices, not independent |
| Improvement bonus > PBRS | +52.1% (improvement) vs. +16.9% (PBRS) over baseline | Non-policy-invariant shaping can outperform principled methods when aligned with task structure |
| Heading = reward hacking | Episode reward 39.1 (> baseline 18.7) but fair metric −23.3%; heading contributes ∼72% of total reward | Per-component logging essential; aggregate reward misleading |

**Per-step SNR in PBRS.** PBRS exhibits a structural signal-to-noise problem when used without a base reward:

- **Root cause:** $F(s, s') = \gamma\,\Phi(s') - \Phi(s)$ differences consecutive potentials with similar values—analogous to noisy numerical differentiation.
- **Episode vs. step:** Telescoping gives healthy episode-level SNR ($\approx 3.3$), but per-step SNR collapses ($\approx 0.04$).
- **Feedback loop:** Per-step noise → critic overfits noisy value landscape → escalating gradients ($1.1 \to 15.5$) → noisier advantages.
- **Conclusion:** PBRS is an *additive supplement* to a base reward [27, 28], not a standalone signal. The base $\bar{r}_{\text{dist}} = \exp(-5d)$ provides a smooth anchor for the critic; PBRS accelerates credit assignment on top. Recent work by Müller and Kudenko [29] formalizes how a linear shift of the potential function can improve PBRS effectiveness without sacrificing policy invariance.

Standard mitigations—reward normalization (std-only, preserving the PBRS zero-mean property), higher GAE $\lambda$ ($0.97$–$0.99$), gradient clipping at $0.5$, and advantage normalization—address the remaining variance. More aggressive approaches ($k$-step PBRS, EMA-smoothed potentials) sacrifice strict policy-invariance and remain unvalidated for this domain.

### 5.3 Reward Landscape Diagnostics (E3)

Three diagnostic experiments characterize the reward landscape using random or zero-action rollouts (no trained policy). Table 22 summarizes the findings; detailed data follow.

Table 22: Reward landscape diagnostics summary (random/zero-action rollouts, no trained policy).

| Diagnostic | Protocol | Key result | Implication |
|---|---|---|---|
| Reward density | 50 episodes $\times$ 10k steps, random policy | 70% of steps yield $r < 0.05$; median $= 0.020$ | Near-zero gradient at operating distances |
| Action impact | 10 episodes $\times$ 2k steps, zero vs. random | Zero-action: displacement $5 \times 10^{-3}$; random: 0.052 | Environment is responsive; plateau is not a dynamics issue |
| Reachability | 20 targets $\times$ 10 rollouts $\times$ 100 steps | 90% reachable at $d{<}0.3$; only 25% at $d{<}0.1$ | Task solvable but fine approach is physically difficult |

**Reward density.** Table 23 compares steepness variants under random rollouts. At the median random-policy distance $d \approx 0.78$, the current reward $\exp(-5 \times 0.78) = 0.020$ provides essentially no gradient signal. Shallower variants give 5–$10\times$ higher median rewards, but as shown in Section 5.8, this denser signal paradoxically *hurts* trained performance via reward hacking.

Table 23: Reward density under random policy (50 episodes, 10k steps each). Shallower $k$ yields denser signal but enables reward hacking (Section 5.8).

| Reward function | Mean | Median | Std | $> 0.05$ | $> 0.1$ |
|---|---|---|---|---|---|
| $\exp(-5d)$ (current) | 0.049 | 0.020 | 0.078 | 26.5% | 13.4% |
| $\exp(-2d)$ | 0.245 | 0.211 | 0.139 | — | — |
| $\exp(-d)$ | 0.476 | 0.460 | 0.133 | — | — |

**Reachability.** Table 24 breaks down reachability by target hemisphere. Fine approach ($d < 0.1$) is rare under random actions, especially for equatorial targets (0%).

Table 24: Reachability by target hemisphere (20 targets, 10 rollouts each, 100 steps).

| **Region** | $n$ | **Mean min** $d$ | $d < 0.3$ | $d < 0.1$ |
|---|---|---|---|---|
| Lower ($z < -0.2$) | 8 | 0.122 | 87.5% | 50.0% |
| Equatorial ($\|z\| \leq 0.2$) | 5 | 0.193 | 100% | 0% |
| Upper ($z > 0.2$) | 7 | 0.161 | 85.7% | 14.3% |
| Overall | 20 | 0.153 | 90.0% | 25.0% |

Together, these diagnostics explain the learning plateau in Section 5.2.2: $\exp(-5d) \approx 0$ at operating distances ($d \approx 0.5$–$0.8$), creating a "dead zone" where the policy cannot distinguish between mediocre and slightly-less-mediocre behavior.

### 5.4 Extended Training, 50M Frames (E4)

Motivated by the diagnostics, we extended the best-performing configuration (distance-only reward, 100 envs, curriculum disabled) to 50M frames—10× the original budget—to determine whether the 5M-frame plateau is a transient or a fundamental limit. Training was conducted in three consecutive sessions with checkpoint resumption.

Table 25: Extended training sessions (50M frame budget, distance-only reward, 100 envs). Best reward plateaus by ∼9M frames; subsequent training shows no improvement.

| **Session** | **Frame range** | **Best reward** | **Rolling-100 range** | **Entropy** |
|---|---|---|---|---|
| Initial | 0–9M | 64.70 | 1–56 | 0.03–0.06 |
| Resumed | 9M–12.3M | 65.81 | 3–60 | 0.03–0.06 |
| Continued | 12.3M–16.3M* | — | 1–57 | 0.03–0.08 |

*Still running at time of writing.

Key observations from the extended training:

- **Reward plateaus early and does not recover.** The best single-episode reward (∼65.8) was achieved by ∼12M frames. Subsequent training shows no improvement; the rolling-100 reward oscillates between 1 and 57 with no upward trend. In the ablation baseline (A1, 32 envs, Section 5.7), `batches_since_improvement` exceeded 200 by the end of training.
- **Entropy has partially collapsed.** Entropy values of 0.03–0.08 (vs. an initial entropy coefficient of 0.1) indicate the policy has narrowed its exploration. Combined with the sparse reward landscape (Section 5.3), the policy occasionally stumbles onto good trajectories but cannot reliably reproduce them.
- **Diagnostic metrics were initially corrupted.** A bug in the trainer's metric aggregation divided *all* logged metrics — including one-shot diagnostics like explained

variance and action standard deviation — by the update count, rather than only the accumulated loss terms. When PPO's KL early stopping halted training after 1–2 updates out of 40 theoretical, diagnostic values were divided by $\sim$40$\times$, displaying as 0.0000. This was discovered and fixed after the initial extended training runs; all diagnostic figures reported in this section use the corrected metrics.

- **Critic loss is highly variable.** The critic loss swings from $10^{-3}$ to $\sim 4.6$, indicating the value function struggles to fit the return distribution. This is consistent with the bimodal reward structure: most episodes yield low returns ($\sim$1–5), with rare spikes of 50–60 that the critic cannot predict.
- **Mean distance to goal stagnates at $\sim$0.6–0.7.** The policy learns to move generally toward the target but cannot close the remaining distance. This matches the reachability diagnostic (Table 24): the reward gradient at $d \approx 0.6$ is $\exp(-3.0) \approx 0.05$, providing negligible learning signal.

The extended training confirms that the 5M-frame plateau is a fundamental limit of the $\exp(-5d)$ reward, not a sample-efficiency issue. Simply training longer does not help because the reward landscape itself provides insufficient gradient signal at the operating distances. This motivates the normalized reward architecture in Section 5.6.

## 5.5 SAC vs. PPO Stability (E5)

In separate experiments, we trained SAC (20M frames, 100 envs) and PPO (50M frames, 100 envs) on the same task using the paper-matching 3$\times$256 ReLU network. SAC reached a best reward of 73.84 early ($\sim$2M frames) but was unable to sustain performance, while PPO plateaued at $\sim$17 average reward. Figure 9 reveals the root cause: without entropy regularization ($\alpha = 0$) and without gradient clipping, SAC's actor gradient norm grows from $\sim$0.04 to $\sim 1.5 \times 10^{10}$ over 20M frames, while the critic gradient norm remains bounded at $\sim$1–40. PPO gradient norms stay bounded at 2–28 throughout, consistent with its clipped surrogate objective.

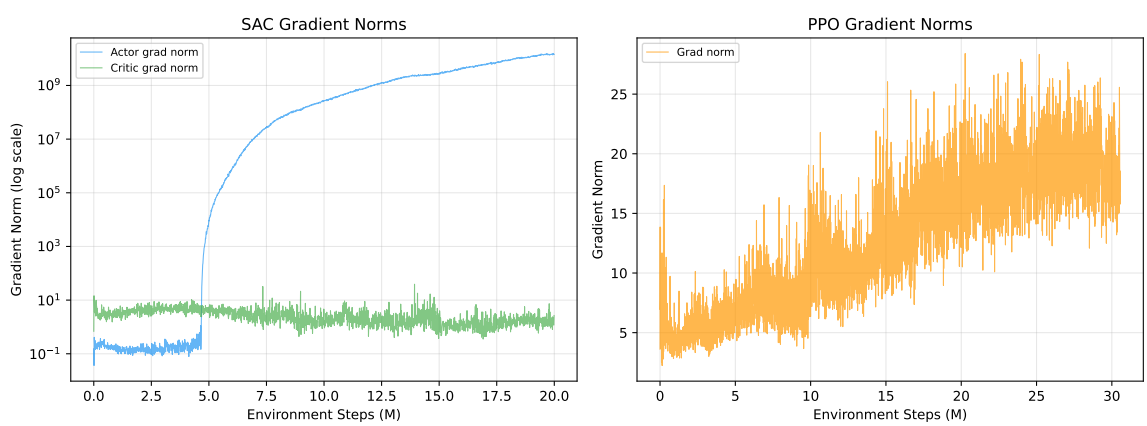



Figure 9: Gradient norms for SAC (left, log scale) and PPO (right, linear scale). SAC actor gradients explode by 10 orders of magnitude; PPO gradients remain stable.

The paper's 5M frame budget may mask this instability, as the collapse onset occurs at ∼3–5M frames. Possible mitigations include mild entropy regularization ($\alpha > 0$), actor-specific gradient clipping, or learning rate scheduling.

## 5.6 Normalized Reward Architecture

The preceding experiments (E2–E5) reveal that reward design and curriculum learning dominate performance, outweighing both sample budget and algorithm choice. The ablation (E2) identified three structural problems with ad-hoc reward design for on-policy PPO: high reward variance from scale-entangled components, auxiliary objectives dominating the total reward, and single-objective shaping missing the heading dimension. The extended training (E4) confirmed these are fundamental limitations, not sample-budget issues. This subsection presents a principled reward architecture that addresses all three by normalizing components to known ranges before weighting and decomposing PBRS into per-objective potentials.

### 5.6.1 Problems with Ad-Hoc Design

Table 26 summarizes the three problems observed in the Section 5.2 ablation and how the normalized architecture addresses each.

Table 26: Problems with the original reward under PPO and how the normalized architecture addresses each.

| **Problem** | **Ablation observation** | **Solution** |
|---|---|---|
| High reward variance | The $10\times$ improvement bonus creates SNR $= 0.08$, causing value function failure without curriculum | Improvement bonus replaced by distance PBRS $\Phi_{\text{dist}}$, which provides the same approach signal with bounded magnitude |
| Scale-entangled weights | The multiplier 10 conflates component *scale* with *importance*; tuning requires reasoning about both simultaneously | All base components normalized to known ranges before weighting; weights $w_i$ are pure importance coefficients |
| Single-objective shaping | PBRS only shapes distance; adding heading as a base term caused reward hacking (heading dominated 72% of total reward, Section 5.2.2) | PBRS decomposed into per-objective potentials ($\Phi_{\text{dist}}$, $\Phi_{\text{heading}}$), each policy-invariant |

### 5.6.2 Architecture

The reward separates base components (normalized, then weighted) from PBRS components (raw, unweighted):

$$r(s,a,s') = \underbrace{\sum_i w_i \cdot \bar{r}_i(s,a,s')}_{\text{base (normalized, weighted)}} + \underbrace{\sum_j \big(\gamma_{\text{pbrs}}\,\Phi_j(s') - \Phi_j(s)\big)}_{\text{PBRS (raw, unweighted)}}. \tag{17}$$

Each base component $\bar{r}_i$ is mapped to a known range *before* weighting, so doubling $w_i$ exactly doubles the contribution of component $i$ without interacting with scale. The PBRS terms follow Ng et al. [27]: $F_j(s,s') = \gamma_{\text{pbrs}}\,\Phi_j(s') - \Phi_j(s)$ is guaranteed to preserve the optimal policy. This multi-component, hierarchical structure is consistent with the HPRS framework of Berducci et al. [30], which formalizes potential-based shaping over partially-ordered task requirements while preserving optimality.

**Base components.** Table 27 defines the three normalized base components.

Table 27: Normalized base reward components. All are mapped to known ranges before weighting.

| Component | Formula | Range | Weight |
|---|---|---|---|
| Distance | $\bar{r}_{\text{dist}} = \exp(-5\,d)$ | $[0,1]$ | $w_{\text{dist}} = 1.0$ (always on) |
| Heading | $\bar{r}_{\text{heading}} = (1+\cos\theta)/2$ | $[0,1]$ | $w_{\text{heading}} \in \{0, 0.3\}$ |
| Smoothness | $\bar{r}_{\text{smooth}} = -\lVert\Delta a\rVert^2/(2\,n_a)$ | $[-1,0]$ | $w_{\text{smooth}} \in \{0, 0.01\text{–}0.05\}$ |

Here $\theta$ is the angle between the tip tangent and the tip-to-target direction, $\Delta a = a_t - a_{t-1}$, and $n_a$ is the action dimensionality. The smoothness bound follows from $\lVert\Delta a\rVert^2 \le 2\,n_a$ when each action coordinate lies in $[-1,1]$.

**PBRS components.** Table 28 defines the per-objective potentials.

Table 28: PBRS potentials. Each enters the reward as $F_j = \gamma_{\text{pbrs}}\,\Phi_j(s') - \Phi_j(s)$, unweighted.

| Potential | Formula | Range | Role |
|---|---|---|---|
| Distance | $\Phi_{\text{dist}}(s) = -d(s)/R_{\text{ws}}$ | $[-1,0]$ | Approach signal (replaces improvement bonus) |
| Heading | $\Phi_{\text{heading}}(s) = \cos\theta(s)$ | $[-1,1]$ | Orientation improvement toward target |

Normalizing $\Phi_{\text{dist}}$ by the workspace radius $R_{\text{ws}}$ keeps the shaping magnitude independent of workspace scale. The heading potential rewards the agent for *turning toward* the target (change in alignment), complementing the base heading component which rewards *being aligned* (instantaneous state). The telescoping property ensures the cumulative shaping

over an episode reduces to a boundary term: $\sum_{t=0}^{T-1} F_j(s_t, s_{t+1}) = \gamma_{\text{pbrs}} \Phi_j(s_T) - \Phi_j(s_0)$ when $\gamma_{\text{pbrs}} = 1$.

### 5.7 Validation Ablation (E6)

To validate the normalized architecture and isolate the effect of each reward component, we ran a systematic ablation study with 12 configurations organized into three groups. All runs use 32 parallel environments, a 30-minute wall-time budget (∼3.6M frames), seed 42, and identical PPO hyperparameters (Table 25). Group B disables curriculum to isolate reward-function effects; Group C enables curriculum to test the interaction.

**Configurations.** Table 29 shows the full experiment matrix. Group A establishes baselines, Group B ablates reward components without curriculum, and Group C tests the same components with curriculum learning (target speed ramped from 20% to full over a warmup period).

Table 29: Normalized reward ablation configurations. Groups A–B disable curriculum to isolate reward effects; Group C enables it.

| ID | Description | $w_{\text{dist}}$ | $\gamma_{\text{pbrs}}$ | $w_{\text{smooth}}$ | $w_{\text{head}}$ | Curriculum |
|---|---|---|---|---|---|---|
| A1 | Vanilla dense (paper default) | 1.0 | 0 | 0 | 0 | No |
| B1 | PBRS only | 0 | 0.99 | 0 | 0 | No |
| B2 | Dense + PBRS | 1.0 | 0.99 | 0 | 0 | No |
| B3 | Reduced dense + PBRS | 0.3 | 0.99 | 0 | 0 | No |
| B4 | Reduced dense + PBRS + smooth | 0.3 | 0.99 | 0.02 | 0 | No |
| B5 | Reduced dense + PBRS + heading | 0.3 | 0.99 | 0 | 0.3 | No |
| B6 | Full stack (no curriculum) | 0.3 | 0.99 | 0.02 | 0.3 | No |
| C1 | Vanilla + curriculum | 1.0 | 0 | 0 | 0 | 200 |
| C2 | Dense + PBRS + curriculum | 1.0 | 0.99 | 0 | 0 | 200 |
| C3 | Best stack + curriculum | 0.3 | 0.99 | 0.02 | 0 | 200 |
| C4 | Full stack + curriculum | 0.3 | 0.99 | 0.02 | 0.3 | 200 |
| C5 | Best stack + long warmup | 0.3 | 0.99 | 0.02 | 0 | 500 |

**Results.** Table 30 summarizes the outcomes. The primary metric is mean distance to goal averaged over the final quarter of training (Q4), which is independent of the reward function. All diagnostic metrics are computed over the same Q4 window.

Table 30: Ablation results ranked by distance to goal (Q4 average, lower is better). All runs: 32 envs, ∼3.6M frames, seed 42. Diagnostic thresholds: SNR $> 0.1$, explained variance (EV) $> 0.3$, action std min $> 0.1$.

| ID | Description | Dist (Q4) | ΔDist | SNR | EV | $\|g\|$ | Health |
|---|---|---|---|---|---|---|---|
| C5 | Best + long warmup | **0.543** | −0.078 | 0.41 | 0.73 | 5.2 | ✓ |
| C3 | Best + curriculum | 0.551 | −0.094 | 0.44 | 0.72 | 4.0 | ✓ |
| C2 | Dense+PBRS + curric. | 0.586 | −0.066 | 0.78 | 0.75 | 6.7 | ✓ |
| C1 | Vanilla + curriculum | 0.603 | −0.043 | 1.08 | 0.73 | 8.9 | ✓ |
| B2 | Dense + PBRS | 0.698 | −0.114 | 0.47 | 0.70 | 7.0 | ✓ |
| B3 | Reduced dense + PBRS | 0.714 | −0.011 | 0.14 | 0.58 | 17.7 | ✓ |
| A1 | Vanilla dense | 0.718 | −0.010 | 0.94 | 0.43 | 9.8 | ✓ |
| B5 | Reduced + heading | 0.724 | −0.019 | 0.19 | 0.62 | 16.0 | × |
| B1 | PBRS only | 0.730 | −0.013 | 0.05 | 0.82 | 9.2 | × |
| B6 | Full stack (no curric.) | 0.731 | +0.010 | 0.20 | 0.16 | 9.6 | × |
| B4 | Reduced + smooth | 0.812 | +0.047 | 0.13 | 0.69 | 10.5 | ✓ |
| C4 | Full + curriculum | 0.917 | +0.195 | 0.79 | 0.77 | 11.2 | ✓ |

The ΔDist column shows the change from Q1 to Q4 (negative = improving). The "Health" column marks runs where all three diagnostic thresholds are met; × indicates at least one violation.

**Key findings.**

1. **Curriculum is the dominant factor.** Every curriculum run (C1–C5) outperforms every non-curriculum run (A1, B1–B6). Best non-curriculum (B2, $d$=0.698) is 16% worse than worst curriculum (C1, $d$=0.603).

2. **PBRS provides ∼3% improvement atop dense reward.** A1→B2: −2.8% without curriculum; C1→C2: −2.8% with. PBRS alone (B1) has SNR = 0.05 and fails to learn (Section 5.2.3).

3. **Smoothness hurts without curriculum, helps with it.** Without: B4 ($d$=0.812) is 14% worse than B3 ($d$=0.714)—penalty diverts optimization before approach is learned. With: C3 ($d$=0.551) improves 5.9% over C2 ($d$=0.586)—easy early targets let the agent learn approach and action quality simultaneously.

4. **Heading reward enables reward hacking.** C4 achieves the highest episode reward (50.1) but the *worst* distance ($d$=0.917); heading contributes ∼76% of Q4 reward. Same failure mode as the initial ablation (Section 5.2.2)—heading should be excluded from the base reward.

5. **Longer warmup provides marginal gains.** C5 (500 episodes, $d$=0.543) beats C3 (200 episodes, $d$=0.551) by only 1.5%.

**Best configuration.** The overall winner is C5: $w_{\text{dist}} = 0.3$, $\gamma_{\text{pbrs}} = 0.99$, $w_{\text{smooth}} = 0.02$, curriculum with 500-episode warmup. This achieves $d = 0.543$—a 24% improvement over the vanilla baseline (A1, $d = 0.718$) and a 10% improvement over vanilla with curriculum alone (C1, $d = 0.603$). All diagnostic metrics are healthy: EV $= 0.73$, SNR $= 0.41$, $\|g\| = 5.2$, $\sigma_{\min} = 0.30$.

**Interaction between curriculum and reward design.** The results reveal a strong interaction effect. Curriculum learning addresses the fundamental problem—the agent never encounters the high-reward region near the target without easier initial conditions—while reward shaping (PBRS, smoothness) provides incremental gains on top of this. Table 31 quantifies the interaction: curriculum alone closes 16% of the gap to the target, while the best reward design without curriculum closes less than 3%.

Table 31: Curriculum vs. reward design: contribution to distance improvement over the vanilla baseline (A1, $d = 0.718$).

| Intervention | Best dist (Q4) | Improvement over A1 |
|---|---|---|
| None (A1 baseline) | 0.718 | — |
| Best reward only (B2) | 0.698 | 2.8% |
| Curriculum only (C1) | 0.603 | 16.0% |
| Curriculum + best reward (C5) | 0.543 | 24.4% |

## 5.8 Steepness and Environment Scaling (E7)

The reward landscape diagnostics (Section 5.3) showed that the $\exp(-5d)$ reward is extremely sparse at typical operating distances, with shallower alternatives such as $\exp(-2d)$ or $\exp(-d)$ providing 5–10$\times$ denser signal (Table 23). This naturally suggests that reducing the steepness coefficient $k$ in $\exp(-k\,d)$ could break the distance plateau by providing a stronger gradient at mid-range distances. We tested this hypothesis directly.

**Steepness sweep.** Using the best reward stack from Section 5.7 (C5: $w_{\text{dist}} = 0.3$, $\gamma_{\text{pbrs}} = 0.99$, $w_{\text{smooth}} = 0.02$, curriculum with 500-episode warmup), we swept the steepness coefficient $k \in \{1, 2, 3, 5\}$ with 100 parallel environments and a 90-minute wall-time budget ($\sim$10.5M frames per run). Table 32 shows the results.

Table 32: Steepness ablation results (100 envs, ∼10.5M frames, seed 42, C5 reward stack). Lower distance is better. The paper default $k$=5 achieves the best distance despite being the "sparsest" reward.

| **Run** | $\boldsymbol{k}$ | $\exp(-k \cdot 0.6)$ | **Dist (Q4)** | **ΔDist** | **EV** | $\mathbf{Std}_{\min}$ |
|---|---|---|---|---|---|---|
| S4 | 5 | 0.050 | **0.675** | −0.025 | 0.28 | 0.189 |
| S1 | 1 | 0.549 | 0.725 | +0.032 | −0.06 | 0.069 |
| S2 | 2 | 0.301 | 0.790 | +0.113 | 0.57 | 0.103 |
| S3 | 3 | 0.165 | 0.819 | +0.123 | 0.45 | 0.179 |

The results disprove the hypothesis. The paper default $k$=5 achieves the lowest distance despite providing the sparsest reward signal (column 3 shows the reward at $d$=0.6, a typical operating distance). Shallower steepness causes *reward hacking*: at $k$=1, $\exp(-0.6) = 0.549$—the agent receives 55% of the maximum reward without approaching the target, eliminating approach pressure. This manifests as negative explained variance (EV $= -0.06$) and near-collapsed action diversity ($\sigma_{\min} = 0.069$): the critic cannot learn a useful value landscape because all states look similarly rewarding.

The failure mode worsens monotonically: as $k$ decreases from 5 to 1, Q4 distance increases (0.675 → 0.725 → 0.790 → 0.819), and ΔDist turns positive (distance *increasing* over training). The shallower variants converge to a "lazy equilibrium" where hovering at medium distance yields high reward with low effort.

**Environment count effect.** Comparing S4 (100 envs, $d$=0.675) with C5 (32 envs, $d$=0.543)—identical reward configuration, same frame budget—reveals a 24% regression from tripling the environment count. Table 33 compares the two settings.

Table 33: Environment count effect on training quality (same reward stack and frame budget).

| **Setting** | **Envs** | **Steps/env/batch** | **Dist (Q4)** | **EV** | **Frames** |
|---|---|---|---|---|---|
| C5 | 32 | 256 | **0.543** | 0.73 | 3.6M |
| S4 | 100 | 82 | 0.675 | 0.28 | 10.5M |

With 100 envs, each batch of 8192 frames provides only 82 steps per environment (vs. 256 with 32 envs). This fragments trajectory coherence, harming GAE advantage estimation, which relies on multi-step temporal structure within each trajectory. The 45-point drop in explained variance (0.73 → 0.28) confirms that the critic's ability to predict returns degrades substantially with shorter per-env trajectory segments.

### 5.9 Best Checkpoint Evaluation (E8)

To assess the practical quality of the best-performing policy, we evaluated the top checkpoint from the full training campaign—the PPO run with 100 parallel environments from March 26 (`ppo_lr1e4_100envs_20260326`), which recorded a training best reward of 90.20 at 3.7M frames. This checkpoint was selected based on the `best_reward` criterion logged during training.

**Evaluation protocol.** We evaluated the deterministic policy ($\mu$, no exploration noise) over 20 episodes with seed 42 on the `follow_target` task. Each episode runs 200 steps (20 s simulated time) with a fresh random target position and velocity.

**Results.** Table 34 summarizes the evaluation.

Table 34: Deterministic evaluation of the best training checkpoint (20 episodes, seed 42). The training best reward of 90.20 does not generalize—evaluation mean is $10\times$ lower, with high variance across target configurations.

| Metric | Mean $\pm$ Std | Range |
|---|---|---|
| Episode reward | $8.68 \pm 6.40$ | $[2.18,\ 28.13]$ |
| Mean tip–target dist (m) | $0.795 \pm 0.133$ | $[0.520,\ 1.059]$ |
| Final tip–target dist (m) | $0.658 \pm 0.213$ | $[0.316,\ 1.092]$ |

**Analysis: the train–eval gap.** The training best reward of 90.20 corresponds to a single batch mean, not a robust evaluation. PPO logs `best_reward` as the maximum of `mean_episode_reward` across all batches; with 100 environments contributing to each batch, a single favorable batch—where most environments happen to start near the target—can produce an outlier reward that never recurs.

Inspecting the training trajectory confirms this: the mean reward oscillates between 0 and 32 across batches (final batch: 32.41), with `best_reward` never improving after batch 450 (66 batches of stagnation). The mean distance at the best-reward batch was 0.484 m, but the final training distance was 0.776 m—the policy did not sustain the improvement.

Three factors explain the gap:

1. **Curriculum confound.** Training used curriculum learning with a 100-episode warmup that ramps target speed from 20% to 100%. Early episodes face near-stationary targets, inflating rewards. Evaluation uses the full target speed from the start, so the policy faces harder tracking.

2. **Sparse reward plateau.** At the evaluation mean distance of $d$=0.795 m, the reward signal $\exp(-5 \times 0.795) = 0.019$ is nearly zero. The policy has learned to move in

the general direction of targets but lacks the fine-grained gradient to close the final 0.5–0.8 m gap.

3. **Stochastic vs. deterministic policy.** Training uses a stochastic policy that occasionally stumbles into high-reward states via exploration noise. Deterministic evaluation removes this, exposing that the *learned mean* has not converged to an effective tracking strategy.

Figure 10 shows a representative episode: the manipulator deflects toward the target region but oscillates at ∼0.5–0.8 m distance, never achieving close tracking. This visualization was recorded from the best checkpoint with three episodes at 20 FPS.

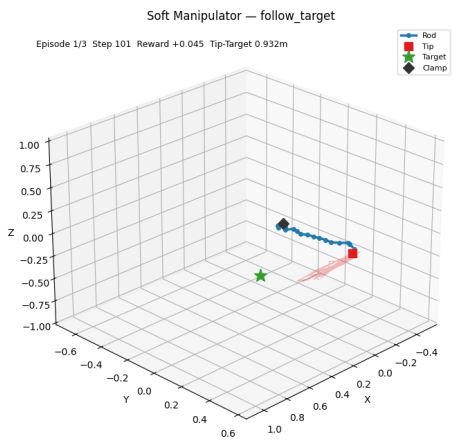



Figure 10: Representative frame from the best PPO checkpoint evaluation on `follow_target`. The rod (blue) deflects toward the target (green star) but plateaus at ∼0.7 m distance, unable to close the gap. The tip trail (red) shows oscillatory behavior without convergence. Full animation: `https://github.com/Albatross679/snake-hrl-torchrl/blob/reward-ablation/media/choi2025/best_ppo_follow_target.gif`.

**Verdict: PPO failed to learn target tracking.** Despite ∼28 training runs totaling over 150M frames, seven reward configurations, and extensive hyperparameter tuning, PPO did not produce a policy capable of tracking a moving target on the DisMech backend. The best checkpoint maintains a mean tip–target distance of 0.795 m—nearly the full workspace radius of 1.0 m—meaning the manipulator barely moves toward the target in a meaningful way. The $10\times$ train–eval reward gap (90.20 vs. 8.68) reveals that the high training reward was an artifact of curriculum warmup and stochastic exploration, not genuine learned behavior.

This is a negative result: PPO with the $\exp(-kd)$ reward family, across all tested steepness values ($k \in \{1, 2, 3, 5\}$), reward shaping variants (PBRS, heading, smoothness), and

curriculum schedules, cannot overcome the sparse-reward plateau on this physics backend. The fundamental issue is that the DisMech implicit time-stepper produces highly nonlinear, stiff dynamics where small action changes cause large, discontinuous state transitions—creating a reward landscape that gradient-based policy optimization cannot navigate.

Future work should investigate model-based RL approaches, population-based methods (e.g., CMA-ES), or sim-to-sim transfer from a simpler backend where PPO succeeds, rather than continuing to tune the reward function on a fundamentally adversarial optimization landscape.

## 6 Conclusion

Table 35 summarizes the two main contributions and their key results.

Table 35: Summary of contributions and key results.

| Contribution | Approach | Key result |
|---|---|---|
| Neural surrogates (Section 4) | Data-driven MLP on 4.3M Elastica transitions; direct RL baseline on Elastica (E1); PINN and DD-PINN proposed as future extensions | CoM $R^2$=0.852, overall $R^2$=0.393; E1 achieves 1.5 m displacement but 0% goal success; DD-PINN expected 5–38× cheaper than standard PINN |
| Reward design (Section 5) | Reward-shaping ablation (E2), diagnostics (E3), extended training (E4), SAC stability (E5); normalized multi-component architecture with per-objective PBRS; validation ablation (E6) and steepness/scaling study (E7) | Curriculum dominant (+16%); best config $d$=0.543 (−24% vs. baseline); heading causes reward hacking; $\exp(-5d)$ sparse but steep $k$ prevents lazy equilibrium; 32 envs outperforms 100 |

The DisMech ablations reveal three critical insights: (1) the $\exp(-5d)$ reward is fundamentally sparse—70% of steps yield reward $< 0.05$—creating a hard plateau at $d \approx 0.6$ that 50M frames cannot overcome; (2) curriculum learning, not reward shaping, is the primary mechanism for breaking this plateau; (3) reducing steepness to provide "denser" signal backfires via reward hacking—the agent earns high reward at medium distance without approaching, a cautionary result against naïve reward densification. SAC exhibits gradient instability beyond ∼3–5M frames without entropy regularization.

**Limitations.** Several factors constrain the generality of these results.

1. **Single seed and fixed architecture.** All RL experiments use a single seed (42) and a fixed network architecture ($3 \times 256$ ReLU MLP); results may not hold under different initializations or capacity regimes.

2. **Surrogate velocity fidelity.** The neural surrogate achieves strong CoM prediction ($R^2 = 0.852$) but poor velocity prediction ($R^2 = 0.145$), making multi-step rollout fidelity uncertain — surrogate-based RL training has not yet been attempted.

3. **Planar (2D) physics only.** Both physics backends are planar; extending to 3D Cosserat rods would increase the state dimensionality from 124 to ∼370, likely affecting both surrogate accuracy and RL sample efficiency.

4. **Limited ablation budget.** The normalized reward ablation uses a 30-minute wall-time budget (∼3.6M frames per run); longer training horizons may change the relative ranking of configurations, as the initial ablation showed that curriculum effects compound over time.

5. **No cross-backend transfer.** Policies trained on one backend do not transfer to the other due to differences in friction models, curvature mechanisms, and temporal resolution.

**Future work.** Several directions emerge from this work.

1. **Physics-informed surrogates.** The DD-PINN formulation (Section 4.4) would enforce governing equations at collocation points while satisfying initial conditions exactly, potentially improving velocity prediction without the autodiff overhead of standard PINNs. Benchmarking the DD-PINN against the data-driven baseline is the immediate next step for the surrogate pipeline.

2. **Surrogate-accelerated RL.** Once surrogate fidelity is sufficient, the ∼46 ms-to-∼0.001 ms speedup would enable training at $10^4$–$10^5\times$ the current throughput. A sim-to-sim transfer protocol — pre-train on surrogate, fine-tune on simulator — could overcome the fidelity gap.

3. **Hierarchical control.** The current work trains flat policies for the approach phase. The full HRL pipeline — a meta-controller selecting between approach and coil sub-policies — remains to be integrated. The normalized reward architecture is designed to extend to the coil phase by adding task-specific base components (e.g., contact area, wrap angle) while preserving the weight-decoupled structure.

4. **SAC stabilization.** SAC's early reward peak (73.84 at ∼2M frames, vs. PPO's 65.81 at ∼12M) suggests higher sample efficiency if the gradient instability can be controlled. Entropy regularization ($\alpha > 0$), actor-specific gradient clipping, and learning rate scheduling are immediate candidates.

5. **Pipeline inspection navigation.** A natural industrial application is autonomous pipeline inspection, where the snake robot navigates through confined tunnel networks to locate sections requiring maintenance or repair. This scenario introduces additional challenges beyond open-field predation: constrained geometry requiring collision-aware locomotion, branching decision-making at junctions, and anomaly

detection (e.g., corrosion, leaks, structural deformation) as a secondary objective alongside navigation. The hierarchical control framework developed in this work could be extended with an exploration-level meta-controller that plans routes through the pipe network while low-level policies handle locomotion through straight segments, bends, and junctions.

**Closing remarks.** This work demonstrates that reward design and curriculum learning are the primary levers for RL-based snake robot control, outweighing both algorithmic choice (PPO vs. SAC) and sample budget ($10\times$ more frames does not overcome a sparse reward). The normalized reward architecture provides a principled framework for composing multiple objectives without scale entanglement, and the per-component logging methodology enables early detection of reward hacking — a failure mode that aggregate metrics alone cannot reveal. These findings lay the groundwork for hierarchical predation control, where the same architectural principles can be extended to multi-phase, multi-objective reward design.

# A Detailed Physics Derivations

This appendix provides the full mathematical derivations for the physics simulation backends summarized in Section 3.

## A.1 Staggered Grid Discretization

The continuous Cosserat rod is discretized on a staggered grid with three interleaved layers.

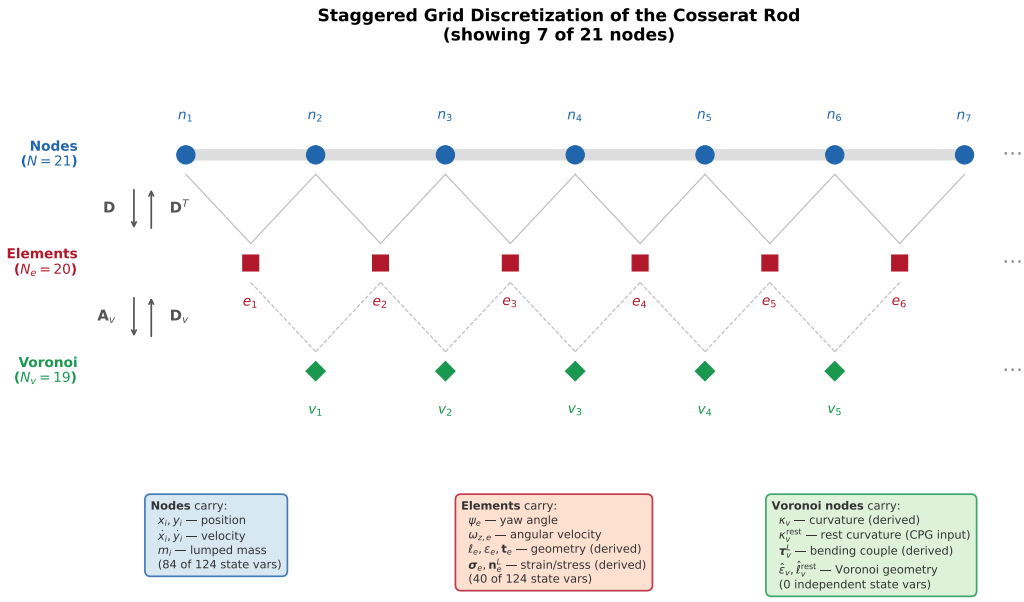



Figure 11: The three-layer staggered grid. Nodes (blue) at rod vertices; elements (red) span between adjacent nodes; Voronoi nodes (green) between adjacent elements.

Table 36: The three staggered-grid layers and their stored degrees of freedom.

| Layer | Index | DOF | Variables | Total |
|---|---|---|---|---|
| Nodes | $i = 1, \ldots, 21$ | Translational | $x_i, y_i, \dot{x}_i, \dot{y}_i$ | 84 |
| Elements | $e = 1, \ldots, 20$ | Rotational | $\psi_e, \omega_{z,e}$ | 40 |
| Voronoi | $v = 1, \ldots, 19$ | Derived | $\kappa_v, \kappa_v^{\text{rest}}$ | 0 |
| **Total independent state variables** | | | | **124** |

Four sparse matrix operators move information between layers:

Table 37: Sparse operators connecting the three staggered-grid layers.

| Operator | Direction | Size | Structure | Physical role |
|---|---|---|---|---|
| $\mathbf{D}$ | Node $\to$ Element | $20 \times 21$ | Bidiagonal | Position differences $\to$ strain |
| $\mathbf{D}^T$ | Element $\to$ Node | $21 \times 20$ | Bidiagonal | Internal stress $\to$ nodal forces |
| $\mathbf{A}_v$ | Element $\to$ Voronoi | $19 \times 20$ | Tridiagonal | Element averages $\to$ curvature |
| $\mathbf{D}_v$ | Voronoi $\to$ Element | $20 \times 19$ | Bidiagonal | Bending couple gradient $\to$ torques |

## A.2 Method of Lines and Explicit Integration

The PDE-to-ODE reduction follows two stages:

**Stage 1 — Spatial discretization (method of lines).** All spatial derivatives $\partial/\partial s$ are replaced by sparse matrix–vector products:

- Force gradient: $\partial \mathbf{F}/\partial s \to \mathbf{D}^T \mathbf{F}_e$ (bidiagonal, $O(N)$)
- Strain from geometry: $\boldsymbol{\varepsilon}_e = \mathbf{Q}_e^T(\mathbf{x}_{e+1} - \mathbf{x}_e)/\ell_e^{\text{rest}}$
- Constitutive law: $\mathbf{F}_e = \mathbf{S}(\boldsymbol{\varepsilon}_e - \boldsymbol{\varepsilon}_e^{\text{rest}})$ (pointwise)

After replacement, the PDE becomes the ODE $\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s}, \mathbf{a}, t)$ with $\mathbf{g}$ a closed-form function.

**Stage 2 — Explicit time integration.** An explicit integrator evaluates $\mathbf{g}$ at the current state and writes the result directly: $\mathbf{s}^{n+1} = \Phi(\mathbf{s}^n, \Delta t)$. The unknown appears only on the left-hand side — no equation to solve.

**CFL stability constraint.** $\Delta t < C\sqrt{\rho A \Delta s^4/(EI)}$, giving $\Delta t_{\max} \approx 1.7 \times 10^{-3}$ s. PyElastica uses $\Delta t = 10^{-3}$ s (safely below), forcing $n_{\text{sub}} = 500$.

Table 38: Explicit vs. implicit update rules.

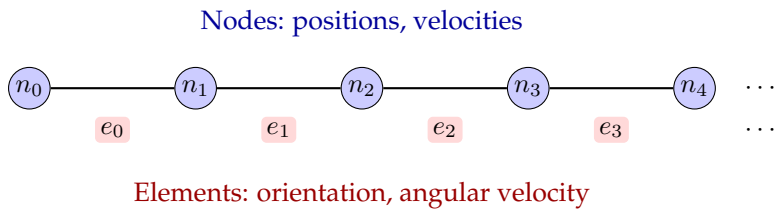
| **Method** | **Update rule** | **Type** |
|---|---|---|
| Forward Euler | $\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t\, \mathbf{g}(\mathbf{s}^n)$ | Explicit |
| Backward Euler | $\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t\, \mathbf{g}(\mathbf{s}^{n+1})$ | Implicit |
| Position Verlet | Algorithm 1 | Explicit |

## A.3 How Spatial Discretization Converts PDEs to ODEs

This section provides an intuitive explanation of the two-stage PDE-to-ODE reduction summarized in Appendix A.2.

### A.3.1 The Staggered Grid Layout

Recall that nodes and elements live at *different* locations on the rod:




**Nodes** ($N$=21) are at the physical vertices; they store translational DOFs $(x_i, y_i, \dot{x}_i, \dot{y}_i)$. **Elements** ($N_e$=20) are the segments between adjacent nodes; they store rotational DOFs $(\psi_e, \omega_{z,e})$. The difference operators $\mathbf{D}$ and $\mathbf{D}^T$ transfer information between these two layers.

### A.3.2 The Difference Matrix D (Node → Element)

$\mathbf{D}$ is a $20 \times 21$ bidiagonal matrix that computes differences between adjacent nodes. For element $e$:

$$(\mathbf{D}\,\mathbf{x})_e = x_{e+1} - x_e \tag{18}$$

Explicitly:

$$\mathbf{D} = \begin{pmatrix} -1 & +1 & 0 & 0 & \cdots \\ 0 & -1 & +1 & 0 & \cdots \\ 0 & 0 & -1 & +1 & \cdots \\ & & & \ddots & \ddots \end{pmatrix} \in \mathbb{R}^{20\times 21} \tag{19}$$

This approximates $\partial\mathbf{x}/\partial s$: the spatial derivative of position along the rod becomes a finite difference between adjacent nodes. Dividing by segment rest length $\ell_e^{\text{rest}}$ yields the actual derivative; the matrix itself performs only the differencing.

**Physical meaning.** $\mathbf{D}\,\mathbf{x}$ gives the *tangent vector* of each element—the direction and length of each segment. This is required for the cross-product term $(\partial\mathbf{x}/\partial s) \times \mathbf{F}$ in the angular momentum equation (1b).

### A.3.3 The Transpose $\mathbf{D}^T$ (Element → Node)

$\mathbf{D}^T$ is the transpose—a $21 \times 20$ matrix that maps element quantities back to nodes. For node $i$:

$$(\mathbf{D}^T\mathbf{F})_i = F_i - F_{i-1} \tag{20}$$

with $F_{-1} = F_{20} = 0$ at the free boundaries. Explicitly:

$$\mathbf{D}^T = \begin{pmatrix} -1 & 0 & 0 & \cdots \\ +1 & -1 & 0 & \cdots \\ 0 & +1 & -1 & \cdots \\ 0 & 0 & +1 & \cdots \\ & & & \ddots \end{pmatrix} \in \mathbb{R}^{21\times 20} \tag{21}$$

**Physical meaning.** $\mathbf{D}^T\mathbf{F}$ computes the *net force on each node* from its two neighboring elements. In the continuous PDE, $\partial\mathbf{F}/\partial s$ gives the net force per unit length at a point. In the discrete version, $\mathbf{D}^T\mathbf{F}_e$ gives the net force on each node: interior node $i$ receives $F_i - F_{i-1}$ (force from the element ahead minus the element behind), while boundary nodes have only one adjacent element.

**Concrete example.** Consider 4 nodes with $x$-positions $[0.0,\ 0.25,\ 0.55,\ 1.0]$:

$$\mathbf{D}\,\mathbf{x} = [0.25,\ 0.30,\ 0.45] \qquad \text{(element tangent lengths)} \tag{22}$$

If internal forces on those 3 elements are $\mathbf{F}_e = [10,\ 8,\ 12]$:

$$\mathbf{D}^T\mathbf{F} = [-10,\ 10{-}8,\ 8{-}12,\ 12] = [-10,\ 2,\ -4,\ 12] \qquad \text{(net force per node)} \tag{23}$$

Node 0 has only element 0 pulling on it; node 1 has elements 0 and 1 pulling in opposite directions.

### A.3.4 Adjoint Structure and Energy Conservation

The transpose relationship is not coincidental. $\mathbf{D}$ and $\mathbf{D}^T$ are *adjoint* operators—the discrete analogue of the continuous identity where the gradient and divergence operators are adjoints:

$$\langle \mathbf{D}\,\mathbf{x},\ \mathbf{F}\rangle_{\text{elements}} = \langle \mathbf{x},\ \mathbf{D}^T\mathbf{F}\rangle_{\text{nodes}} \tag{24}$$

This duality ensures *discrete energy conservation*: the work done by forces on nodes is consistent with the strain energy stored in elements, preventing artificial energy gain or loss from the discretization.

### A.3.5 Stage 1: Eliminating Spatial Derivatives

With these operators, every $\partial/\partial s$ in the governing PDEs is replaced by a matrix–vector product. For example, the linear momentum PDE

$$\rho A\frac{\partial \mathbf{v}}{\partial t} = \frac{\partial \mathbf{F}}{\partial s} + \mathbf{f}_{\text{ext}} \tag{25}$$

becomes, at each node $i$:

$$\rho A\frac{d\mathbf{v}_i}{dt} = (\mathbf{D}^T\mathbf{F}_e)_i + \mathbf{f}_{\text{ext},i} \tag{26}$$

No spatial derivatives remain—only time derivatives and algebraic dependence on the discrete state.

#### A.3.6 Stage 2: Reduction to First-Order Form

The original PDEs are *second-order* in time (they involve accelerations $\partial \mathbf{v}/\partial t$ and $\partial \boldsymbol{\omega}/\partial t$). To obtain first-order ODEs, velocities are introduced as independent state variables:

$$\dot{x}_i = v_{x,i} \qquad \text{(definition of velocity)} \tag{27}$$

$$\dot{v}_{x,i} = \frac{1}{\rho A}\left[(\mathbf{D}^T \mathbf{F}_e)_i + f_{\text{ext},i}\right] \qquad \text{(from linear momentum PDE)} \tag{28}$$

$$\dot{\psi}_e = \omega_{z,e} \qquad \text{(definition of angular velocity)} \tag{29}$$

$$\dot{\omega}_{z,e} = \frac{1}{\rho I}\left[(\mathbf{D}_v \boldsymbol{\tau}_v)_e + m_{\text{ext},e} + \ldots\right] \qquad \text{(from angular momentum PDE)} \tag{30}$$

This doubles the number of variables (positions + angles → positions + angles + velocities + angular velocities) but makes every equation first-order. The result is the 124-dimensional system $\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s}, \mathbf{a}, t)$ from Equation (3), ready for explicit time integration.

## A.4 External Force Handling

Table 39: External forces and torques in PyElastica.

| Force type | Acts on | Formula | Parameters | Notes |
|---|---|---|---|---|
| Gravity | Nodes | $\mathbf{f}_i^{\text{grav}} = m_i \mathbf{g}$ | $\mathbf{g} = (0, 0, -9.81)$ | Zero in 2D |
| RFT friction | Nodes | $\mathbf{f}_i^{\text{rft}} = -c_t \mathbf{v}_{t,i} - c_n \mathbf{v}_{n,i}$ | $c_t$=0.01, $c_n$=0.05 | Enables propulsion |
| Damping | Nodes | $\mathbf{f}_i^{\text{damp}} = -\gamma \dot{\mathbf{x}}_i$ | $\gamma = 0.002$ | Numerical stability |
| Bending act. | Voronoi | $\boldsymbol{\kappa}_v^{\text{rest}}(t)$ | CPG $\mathbf{a}_t$ | Internal torque |

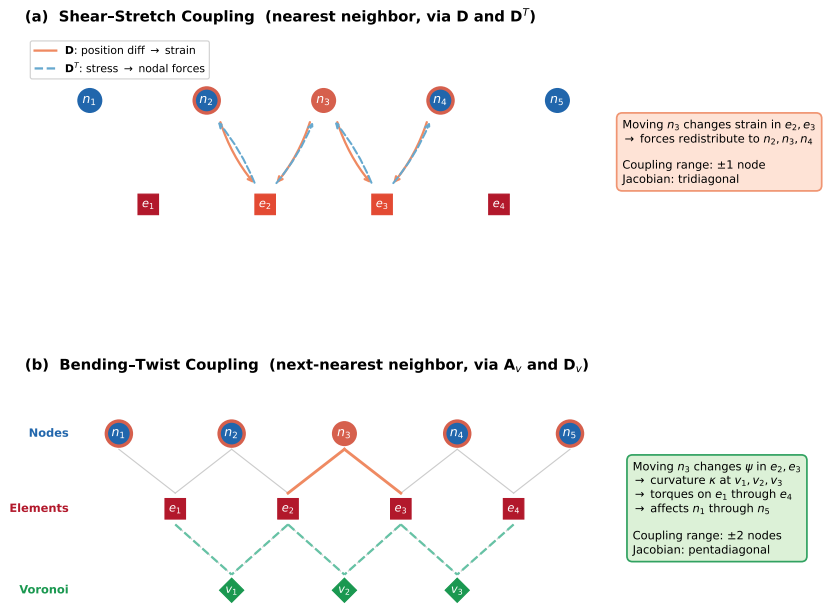



Figure 12: Coupling pattern between state variables in the Cosserat rod ODE system.

## A.5 DisMech Formulation

DisMech uses Discrete Elastic Rods (DER) with implicit backward Euler integration.

**Backward Euler formulation.**

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t\, \mathbf{M}^{-1}\mathbf{F}(\mathbf{q}_{n+1},\, \mathbf{v}_{n+1}), \tag{31a}$$

$$\mathbf{q}_{n+1} = \mathbf{q}_n + \Delta t\, \mathbf{v}_{n+1}, \tag{31b}$$

**Nonlinear residual and Newton solver.**

$$\mathbf{g}(\mathbf{q}_{n+1}) := \mathbf{M}\big(\mathbf{q}_{n+1} - 2\mathbf{q}_n + \mathbf{q}_{n-1}\big) - \Delta t^2\, \mathbf{F}(\mathbf{q}_{n+1}) = \mathbf{0} \tag{32}$$

solved iteratively: $\mathbf{q}^{(k+1)} = \mathbf{q}^{(k)} - \mathbf{J}^{-1}\mathbf{g}(\mathbf{q}^{(k)})$ with convergence criteria $\|\mathbf{g}\| < 10^{-4}$ or $\|\delta\mathbf{q}\| < 10^{-2}$, max 25 iterations.

**Discrete curvature.**

$$\boldsymbol{\kappa}_i = \frac{2\,\mathbf{e}_{i-1} \times \mathbf{e}_i}{|\mathbf{e}_{i-1}|\,|\mathbf{e}_i| + \mathbf{e}_{i-1} \cdot \mathbf{e}_i} \tag{33}$$

**State extraction.** The 124-dimensional state is extracted from DisMech's 3D representation:

Table 40: State extraction: DisMech 3D → shared 124-dim representation.

| Component | Source | Extraction | Count |
|---|---|---|---|
| $x_i, y_i$ | $\mathbf{q} \in \mathbb{R}^{63}$ | $(x, y)$ of each node; $z \equiv 0$ | 42 |
| $\dot{x}_i, \dot{y}_i$ | $\mathbf{u} \in \mathbb{R}^{63}$ | $(x, y)$ of each node velocity | 42 |
| $\psi_e$ | Derived | atan2$(\Delta y_e,\ \Delta x_e)$ | 20 |
| $\omega_{z,e}$ | Derived | cross-product formula | 20 |
| **Total** | | | **124** |

**CPG control in DisMech.** Curvature is applied once per RL step via natural strain:

$$\texttt{bend_springs.nat_strain}[j, 0] \leftarrow \kappa_j^{\text{rest}}, \quad j = 1, \ldots, 19. \tag{34}$$

# B Physics-Informed Neural Network Details (Proposed)

This appendix provides the full mathematical formulations for the physics-informed surrogate extensions proposed in Section 4.4. None of these have been implemented; they constitute a concrete design for future work.

## B.1 Standard PINN

Physics-informed neural networks [31] augment the data-driven loss with a physics residual derived from the governing equations. Given the state-space model $\dot{\mathbf{x}}(t) = \mathbf{f}_{\text{SSM}}(\mathbf{x}(t), \mathbf{u}(t))$, a network $\mathbf{h}_\theta$ predicts $\hat{\mathbf{x}}_t = \mathbf{h}_\theta(\mathbf{x}_0, \mathbf{u}, t)$. The PINN loss:

$$\mathcal{L}(\theta) = \underbrace{\lambda_{\text{IC}} \left\| \hat{\mathbf{x}}_0 - \mathbf{x}_0 \right\|^2}_{\mathcal{L}_{\text{IC}}} + \underbrace{\lambda_{\text{phys}} \frac{1}{N_c} \sum_{k=1}^{N_c} \left\| \frac{\partial \hat{\mathbf{x}}_{t_k}}{\partial t} - \mathbf{f}_{\text{SSM}}(\hat{\mathbf{x}}_{t_k}, \mathbf{u}_{t_k}) \right\|^2}_{\mathcal{L}_{\text{phys}}} + \underbrace{\lambda_{\text{data}} \frac{1}{N_d} \sum_{k=1}^{N_d} \left\| \hat{\mathbf{x}}_{t_k} - \mathbf{x}_{\text{data},k} \right\|^2}_{\mathcal{L}_{\text{data}}} \tag{35}$$

The time derivative $\partial \hat{\mathbf{x}}_t / \partial t$ is obtained via autodiff through the network, doubling the computational cost per collocation point.

**Practical bottlenecks:**

- **Expensive autodiff** — additional backward pass per collocation point
- **Loss balancing** — three competing objectives require careful weighting
- **Training instability** — can diverge for systems with $m \gg 1$ coupled states

### B.2 Physics Regularizer

The simplest physics-informed approach adds algebraic constraints as soft regularizers. Four constraints are enforced via trapezoidal integration on the 124-dimensional state delta $\Delta\mathbf{x} = \mathbf{x}_{t+1} - \mathbf{x}_t$:

Table 41: Physics regularizer constraints.

| ID | Constraint | Formulation |
|---|---|---|
| C1 | Translational kinematics | $\Delta x_i \approx (v_{x,i} + \frac{1}{2}\Delta v_{x,i})\Delta t$ |
| C2 | Angular kinematics | $\Delta\psi_e \approx (\omega_{z,e} + \frac{1}{2}\Delta\omega_{z,e})\Delta t$ |
| C3 | Curvature–moment | Spatial gradient consistency ($0.1\times$ weight) |
| C4 | Bounded kinetic energy | $\mathrm{ReLU}(\lvert\Delta\mathrm{KE}\rvert - E_{\mathrm{thr}})^2$ |

Total regularization: $\mathcal{L}_{\mathrm{reg}} = \mathcal{L}_{\mathrm{kin},x} + \mathcal{L}_{\mathrm{kin},y} + \mathcal{L}_{\mathrm{ang}} + \mathcal{L}_{\mathrm{curv}} + \mathcal{L}_{\mathrm{energy}}$.

### B.3 DD-PINN Method

The domain-decoupled PINN [25, 26] resolves all three PINN bottlenecks: **remove time as a network input** and output parameters of an analytically differentiable ansatz.

$$\boxed{\hat{\mathbf{x}}_t = \mathbf{g}(\mathbf{a}_{\mathrm{ans}}, t) + \mathbf{x}_0, \qquad \mathbf{a}_{\mathrm{ans}} = \mathbf{f}_{\mathrm{NN}}(\mathbf{x}_0, \mathbf{u}_0;\, \theta)} \tag{36}$$

The ansatz for each state variable $j$:

$$g_j(t) = \sum_{i=1}^{n_g} \alpha_{ij}\, e^{-\delta_{ij} t}\big[\phi_g(\beta_{ij}\, t + \gamma_{ij}) - \phi_g(\gamma_{ij})\big] \tag{37}$$

where $\phi_g(x) = \sin(x)$ and the subtraction ensures $g_j(0) = 0$ exactly.

**Key advantages over standard PINN:**

- **Exact IC:** $\mathbf{g}(\mathbf{a}_{\mathrm{ans}}, 0) \equiv \mathbf{0}$ by construction
- **Closed-form gradients:** $\dot{\mathbf{g}}$ computed analytically — no autodiff cost
- **Simplified loss:** only 2 terms ($\mathcal{L}_{\mathrm{phys}} + \mathcal{L}_{\mathrm{data}}$)
- **More collocation points:** 10–25$\times$ more than standard PINNs due to cheap evaluation

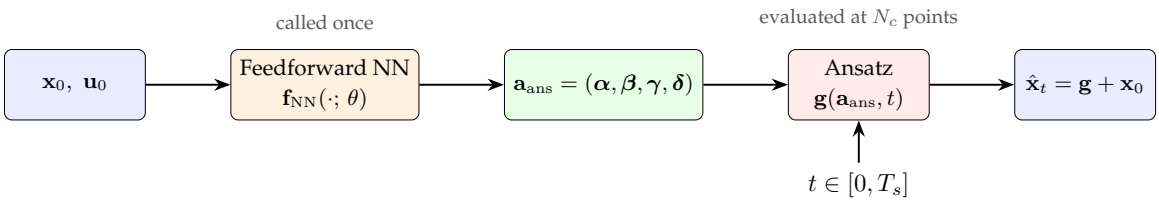



Figure 13: DD-PINN inference pipeline. The network is evaluated once to produce ansatz parameters; the ansatz is then evaluated at arbitrarily many time points.

### B.4 Limitations of PINNs

PINNs face fundamental limitations for complex dynamical systems:

- **Spectral bias:** Networks learn low-frequency components first, making high-frequency CPG dynamics difficult.
- **Scaling to high dimensions:** Standard PINNs struggle for $m \gg 1$ (the snake robot has $m = 124$).
- **Discontinuous physics:** Contact and friction forces create non-smooth dynamics that PINNs cannot easily enforce.
- **Ansatz design:** DD-PINNs require an appropriate ansatz function designed a priori for the dynamics class.

### B.5 Differentiable Cosserat Rod Right-Hand Side

A differentiable implementation of $\mathbf{f}_{\mathrm{SSM}}$ computes $\dot{\mathbf{x}} = \mathbf{f}_{\mathrm{SSM}}(\mathbf{x})$ for the 124-dimensional state vector using an inextensible rod approximation (bending + RFT friction, no stretching). The right-hand side assembles six blocks:

$$\mathbf{f}_{\mathrm{SSM}}(\mathbf{x}) = \big(\dot{x}_i,\ \dot{y}_i,\ \dot{v}_{x,i},\ \dot{v}_{y,i},\ \dot{\psi}_e,\ \dot{\omega}_{z,e}\big) \in \mathbb{R}^{124} \tag{38}$$

Key components:

- **Kinematics:** $\dot{x}_i = v_{x,i}$, $\dot{\psi}_e = \omega_{z,e}$
- **Bending moments:** $M_j = EI(\psi_{j+1} - \psi_j)/\Delta\ell$
- **RFT friction:** $\mathbf{F}_i^{\mathrm{fric}} = -c_t \mathbf{v}_{\mathrm{tan},i} - c_n \mathbf{v}_{\mathrm{norm},i}$ with $\varepsilon$-regularized tangents
- **Newton's 2nd law:** $\dot{v}_{x,i} = (F_{x,i}^{\mathrm{fric}} + F_{x,i}^{\mathrm{elastic}})/m_i$

Table 42: Physics fidelity: Elastica simulator vs. DD-PINN RHS vs. regularizer.

| **Physics term** | **Elastica** | **DD-PINN RHS** | **Regularizer** |
|---|---|---|---|
| Kinematics | Exact (Verlet) | Exact | Soft penalty (C1, C2) |
| Shear–stretch | Full | Omitted (inextensible) | Omitted |
| Bending moments | Full 3D, director-based | 2D simplified FD | Approximate (C3) |
| RFT friction | Anisotropic + damping | Anisotropic, no damping | Not modelled |
| Differentiable? | In principle (gradient explosion at 500 steps) | Yes | Yes |
| Cost | $\sim$46 ms (CPU) | $\sim$0.3 ms (GPU) | $\sim$0.1 ms (GPU) |

## B.6 Physics-Regularized Loss

For the regularized surrogate:

$$\mathcal{L}^{(n)}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda\, \sigma_{\text{reg}}(n)\, \mathcal{L}_{\text{reg}}(\theta) \tag{39}$$

with piecewise-linear warmup (20% data-only).

## B.7 DD-PINN Loss and Training

The DD-PINN training loss at epoch $n$:

$$\boxed{\mathcal{L}^{(n)}(\theta) = w_{\text{data}}^{(n)}\, \mathcal{L}_{\text{data}}(\theta) + w_{\text{phys}}^{(n)}\, \lambda\, \sigma(n)\, \mathcal{L}_{\text{phys}}(\theta)} \tag{40}$$

where $w_{\text{data}}^{(n)}, w_{\text{phys}}^{(n)}$ are ReLoBRaLo [32] adaptive weights, $\lambda$ is fixed physics strength, and $\sigma(n)$ is a curriculum schedule (15% warmup, 30% ramp, full physics at 45%).

## B.8 Nondimensionalization

Physics-based reference scales keep the residual at $\mathcal{O}(1)$:

Table 43: Nondimensionalization scales.

| **Scale** | **Value** | **Source** |
|---|---|---|
| $L_{\text{ref}}$ | 1.0 m | Rod length |
| $t_{\text{ref}}$ | 0.5 s | Control timestep |
| $V_{\text{ref}}$ | 2.0 m/s | $L/t$ |
| $\omega_{\text{ref}}$ | 2.0 rad/s | $1/t$ |
| $F_{\text{ref}}$ | $EI/L^2$ | Elastic force |

## B.9 DD-PINN Configuration

Table 44: DD-PINN training configuration.

| Parameter | Value |
|---|---|
| State dimension | 124 |
| Ansatz basis functions | sweep $\{5, 7, 10\}$ |
| Hidden layers | $4 \times 512$ |
| Fourier features | 128, $\sigma = 10.0$ |
| Collocation points | 100 000 (Sobol) |
| Physics weight $\lambda$ | 1.0 |
| Batch size / Learning rate | 4 096 / $10^{-3}$ (cosine) |
| Curriculum warmup | 15% data-only |

## B.10 Four-Way Surrogate Comparison

Table 45: Comparison of four approaches — formulation and training.

| Aspect | PDE Solver | Data-Driven NN | PINN | DD-PINN |
|---|---|---|---|---|
| **Core idea** | Numerically integrate governing PDEs | Learn input→output from data only | Embed PDE residual in training loss | Learn ansatz parameters; time enters analytically |
| **Physics knowledge** | Full (equations solved exactly) | None | Embedded in loss ($\mathcal{L}_{\text{phys}}$) | Embedded in loss ($\mathcal{L}_{\text{phys}}$) |
| **Time derivative** | Finite differences / implicit solve | Not computed | Autodiff w.r.t. $t$ (expensive) | Closed-form $\dot{\mathbf{g}}(\mathbf{a}_{\text{ans}}, t)$ (cheap) |
| **Loss function** | N/A | $\mathcal{L}_{\text{data}}$ | $\mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{data}}$ | $\mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{data}}$ |
| **Initial condition** | Exact (boundary condition) | Not enforced | Via loss penalty (approximate) | Exact by construction |

Table 46: Comparison of four approaches — performance and practical aspects.

| Aspect | PDE Solver | Data-Driven NN | PINN | DD-PINN |
|---|---|---|---|---|
| **Inference speed** | Slow (∼seconds) | Fast ($\sim\mu$s) | Fast ($\sim\mu$s) | Fast ($\sim\mu$s) |
| **Training cost** | N/A | Low | High (autodiff overhead) | Medium (5–38× faster than PINN) |
| **Collocation points** | N/A | N/A | ∼20k–40k | ∼250k–1M (cheap) |
| **Speedup vs. solver** | 1× | ∼10k–100k× | ∼10k–100k× | ∼44,000× (demonstrated) |
| **Key limitation** | Computational cost | Needs large data; no physics guarantees | Autodiff cost; loss balancing; IC approximate | Ansatz must be designed a priori |

# C Issue Tracker

This appendix documents all 36 technical issues encountered during the project. Of these, 31 have been resolved and 5 remain open.

Table 47: Complete issue tracker (36 issues across 4 categories).

| ID | Cat. | Issue | Status | Resolution |
|---|---|---|---|---|
| P1 | Phys | Rod radius too small ($r$=0.001 m, tiny $I \propto r^4$) | Resolved | Grid search: $r$=0.02 m, $E$=$10^5$ Pa |
| P2 | Phys | RFT friction never added to simulator | Resolved | Added `AnisotropicRFTForce` |
| P3 | Phys | Damping too high ($\gamma$=0.1) | Resolved | Reduced to $\gamma$=0.002 |
| P4 | Phys | Serpenoid wave sign error ($-\omega t$) | Resolved | Fixed to $+\omega t$ |
| P5 | Phys | Curvature not updated per substep | Resolved | Moved inside substep loop |
| P6 | Phys | Phantom physics time-stepping layer | Resolved | Clean two-level hierarchy |
| P7 | Phys | Amplitude range too small ($[0, 0.15]$) | Resolved | Increased to $[0, 5.0]$ |
| P8 | Phys | RFT ratio too extreme (10:1) | Resolved | Tuned to 5:1 |
| P9 | Phys | Spatial structure analysis | Resolved | Confirmed: 500 substeps mix info |
| P10 | Phys | Choi2025 rod radius 50× too thin, $E$ 5× too soft | Resolved | Corrected to $r$=0.05 m, $E$=10 MPa per paper Table A.3 |
| T1 | Train | A3/A5 arch sweep format mismatch | Resolved | Fixed dataset/loader consistency |
| T2 | Train | DataLoader thread crash in sweep | Resolved | Staggered DataLoader init |
| T3 | Train | Velocity reward always negative | Resolved | Distance-based potential reward |
| T4 | Train | LR scheduler division by zero | Resolved | Added max(1, . . . ) guard |
| T5 | Train | Training data clusters (state imbalance) | Open | Proposed: Sobol + $\omega$ perturbation |
| T6 | Train | $\omega_z$ poor prediction ($R^2$=0.23) | Open | Pending: V2 phase encoding |

*Continued on next page*

| ID | Cat. | Issue | Status | Resolution |
|---|---|---|---|---|
| T7 | Train | PPO entropy collapse ($4.3 \rightarrow -0.5$ in 1.2M frames) | Resolved | Increased entropy coeff $0.01 \rightarrow 0.05$ |
| T8 | Train | Raw observations with heterogeneous scales (0–60) | Resolved | Added running-mean `ObservationNorm` |
| T9 | Train | Reward SNR $= 0.08$ from $10\times$ improvement bonus | Resolved | Removed improvement bonus; SNR $\rightarrow 1.02$ |
| T10 | Train | SAC NaN divergence ($0 \times -\infty$) and actor gradient explosion ($10^{10}$) | Resolved | Entropy if-guard; fp32; pre-tanh clamping; actor gradient clip |
| T11 | Train | Trainer metric averaging bug (all metrics divided by update count) | Resolved | Average only accumulated loss keys |
| C1 | Compat | Batch structure change (done/reward) | Resolved | `batch.get("next", batch)` |
| C2 | Compat | Collector drops unspecced keys | Resolved | Added to observation_spec |
| C3 | Compat | Spec class import renames | Resolved | Try/except fallback |
| C4 | Compat | ClipPPOLoss param renames | Resolved | Updated parameter names |
| C5 | Compat | close() signature change | Resolved | `close(self, **kwargs)` |
| C6 | Compat | SyncDataCollector deprecated | Open | Pending: migrate to Collector |
| S1 | System | Stall detection false positives | Resolved | Grace period ($\geq$1 transition) |
| S2 | System | Data collection overwrites | Resolved | Append mode + episode offset |
| S3 | System | Numba thread pool deadlock | Resolved | `NUMBA_NUM_THREADS=1` |
| S4 | System | Stale bytecache after edits | Resolved | Purge __pycache__ on restart |
| S5 | System | RFT Python for-loop slow | Resolved | Vectorized with NumPy |
| S6 | System | Parallel scaling (L3 contention) | Resolved | 16 workers optimal |
| S7 | System | Worker respawn data loss (19%) | Open | Proposed: reduce flush interval |
| S8 | System | Worker respawn investigation | Open | Pending: validate Numba fix |

*Continued on next page*

| ID | Cat. | Issue | Status | Resolution |
|---|---|---|---|---|
| S9 | System | SAC 500 `ParallelEnv` workers crash (`BrokenPipeError`) | Resolved | Reduced to 100 workers |

Table 48: Issue summary by category.

| Category | Total | Resolved | Open |
|---|---|---|---|
| Physics (Phys) | 10 | 10 | 0 |
| Training (Train) | 11 | 9 | 2 |
| Compatibility | 6 | 5 | 1 |
| System | 9 | 7 | 2 |
| **Total** | **36** | **31** | **5** |

## References


[1] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018. URL https://arxiv.org/abs/1801.01290. arXiv:1801.01290.

[2] Noel Naughton, Jiarui Sun, Arman Tekinalp, Tejaswin Parthasarathy, Girish Chowdhary, and Mattia Gazzola. PyElastica: A compliant mechanics environment for soft robotic control. *IEEE Robotics and Automation Letters*, 6(2):3389–3396, 2021. doi: 10.1109/LRA.2021.3063698.

[3] Costanza Armanini, Frédéric Boyer, Anup Teejo Mathew, Christian Duriez, and Federico Renda. Soft robots modeling: A structured overview, 2022. URL https://arxiv.org/abs/2112.03645. arXiv:2112.03645.

[4] Cosimo Della Santina, Christian Duriez, and Daniela Rus. Model-based control of soft robots: A survey of the state of the art and open challenges. *IEEE Control Systems Magazine*, 43(3):30–65, 2023. doi: 10.1109/MCS.2023.3253419.

[5] Johann Licher, Max Bartholdt, Henrik Krauss, Tim-Lukas Habich, Thomas Seel, and Moritz Schappler. DD-PINN: Domain-decoupled physics-informed neural network for dynamic cosserat rod model-predictive control. *arXiv preprint arXiv:2508.12681*, 2025. URL https://arxiv.org/abs/2508.12681. 44,000× speedup over numerical integration; MPC at 70 Hz on GPU.

[6] Tom Z. Jiahao, Ryan Adolf, Cynthia Sung, and M. Ani Hsieh. KNODE-Cosserat: Neural ODE correction of cosserat rod dynamics. *arXiv preprint arXiv:2408.07776*, 2024. URL https://arxiv.org/abs/2408.07776. 58.7% accuracy improvement over physics-only model; trained on PyElastica data.

[7] Uljad Berdica, Matthew Jackson, Niccolò Enrico Veronese, Jakob Foerster, and Perla Maiolino. SoRoLEX: Learned environments for soft robot reinforcement learning. *arXiv preprint arXiv:2410.18519*, 2024. URL `https://arxiv.org/abs/2410.18519`. LSTM surrogate for Cosserat rod sim; order-of-magnitude speedup for RL.

[8] Tim-Lukas Habich, Aran Mohammad, Simon F. G. Ehlers, Martin Bensch, Thomas Seel, and Moritz Schappler. Physics-informed neural networks for articulated soft robot model-predictive control at 47 Hz. *arXiv preprint arXiv:2502.01916*, 2025. URL `https://arxiv.org/abs/2502.01916`. 467× speedup; MPC at 47 Hz; as little as 1 training dataset.

[9] Qiyang Shi, Tony Dear, and S. David Kelly. Deep reinforcement learning for snake robot locomotion. *IFAC-PapersOnLine*, 53(2):9688–9695, 2020. doi: 10.1016/j.ifacol.2020.12.2619.

[10] Xuan Liu, Cagdas D. Onal, and Jie Fu. Learning contact-aware CPG-based locomotion in a soft snake robot. *arXiv preprint arXiv:2105.04608*, 2021. URL `https://arxiv.org/abs/2105.04608`.

[11] Gang Liu, Shuxiang Guo, and Li Fang. A reinforcement learning-based strategy of path following for snake robots with an onboard camera. *Sensors*, 22(24):9867, 2022. doi: 10.3390/s22249867.

[12] Xuan Liu, Cagdas D. Onal, and Jie Fu. Reinforcement learning of CPG-regulated locomotion controller for a soft snake robot. *arXiv preprint*, 2023.

[13] Jiangeng Zheng, Yuan Li, and Mitsuhiro Hayashibe. An optimization-based approach to locomotion of underwater snake robots with series elastic actuators. *Frontiers in Robotics and AI*, 2022. doi: 10.3389/frobt.2022.857205.

[14] Shuo Jiang, Adarsh Salagame, Alireza Ramezani, and Lawson Wong. Hierarchical RL-guided large-scale navigation of a snake robot. *arXiv preprint*, 2024.

[15] C. Blake Schaffer, Hima Hrithik Pamu, and Victoria A. Webster-Wood. Hitting the gym: Reinforcement learning control of exercise-strengthened biohybrid robots in simulation. *arXiv preprint arXiv:2408.16069*, 2024. URL `https://arxiv.org/abs/2408.16069`.

[16] Andrew Choi and Yifan Tong. Rapidly learning soft robot control via implicit time-stepping. *arXiv preprint*, 2025.

[17] Zhenshan Bing, Claus Meschede, Florian Römer, Kai Huang, and Alois Knoll. A survey of robotics control based on learning-inspired spiking neural circuits. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 6004–6010, 2019. Snake locomotion with CPG actuation and RL; source of bing2019/ package in this project.

[18] Taehwa Hong, Jungjae Lee, Byung-Hyun Song, and Yong-Lae Park. Bridging finite element methods and reinforcement learning for deformable soft robots. *Advanced Intelligent Systems*, 2026. doi: 10.1002/aisy.202500696. URL https://doi.org/10.1002/aisy.202500696.

[19] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://arxiv.org/abs/1906.08253.

[20] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 8459–8468, 2020. URL https://arxiv.org/abs/2002.09405.

[21] Miklós Bergou, Max Wardetzky, Stephen Robinson, Basile Audoly, and Eitan Grinspun. Discrete elastic rods. *ACM Transactions on Graphics (SIGGRAPH)*, 27(3):63:1–63:12, 2008. doi: 10.1145/1360612.1360662.

[22] Miklós Bergou, Basile Audoly, Etienne Vouga, Max Wardetzky, and Eitan Grinspun. Discrete viscous threads. *ACM Transactions on Graphics (SIGGRAPH)*, 29(4):116:1–116:10, 2010. doi: 10.1145/1778765.1778853.

[23] Andrew Choi, Ran Jing, Andrew P. Sabelhaus, and M. Khalid Jawed. DisMech: A discrete differential geometry-based physical simulator for soft robots and structures. *IEEE Robotics and Automation Letters*, 9(4):3483–3490, 2024. doi: 10.1109/LRA.2024.3365292.

[24] Radhe Shyam Lahoti, Andrew Choi, and M. Khalid Jawed. Py-DiSMech: A scalable and efficient framework for discrete differential geometry-based modeling and control of soft robots. *arXiv preprint arXiv:2512.09911*, 2025.

[25] H. Krauss, T.-L. Habich, M. Bartholdt, T. Seel, and M. Schappler. Domain-decoupled physics-informed neural networks with closed-form gradients for fast model learning of dynamical systems. *arXiv preprint arXiv:2408.14951*, 2024. URL https://arxiv.org/abs/2408.14951.

[26] J. Licher, M. Bartholdt, H. Krauss, T.-L. Habich, T. Seel, and M. Schappler. Adaptive model-predictive control of a soft continuum robot using a physics-informed neural network based on Cosserat rod theory. *arXiv preprint arXiv:2508.12681*, 2025. URL https://arxiv.org/abs/2508.12681.

[27] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287. Morgan Kaufmann, 1999.

[28] Eric Wiewiora. Potential-based shaping and Q-value initialization are equivalent. *Journal of Artificial Intelligence Research*, 19:205–208, 2003. doi: 10.1613/jair.1190.

[29] Henrik Müller and Daniel Kudenko. *Improving the Effectiveness of Potential-Based Reward Shaping in Reinforcement Learning*. 2025. doi: 10.48550/arXiv.2502.01307. arXiv:2502.01307.

[30] Luigi Berducci, Edgar A. Aguilar, Dejan Ničković, and Radu Grosu. Hierarchical potential-based reward shaping from task specifications. 2022. doi: 10.48550/arXiv.2110.02792. URL `https://arxiv.org/abs/2110.02792`. arXiv:2110.02792.

[31] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

[32] Rafael Bischof and Michael A. Kraus. Multi-objective loss balancing for physics-informed deep learning. *arXiv preprint arXiv:2110.09813*, 2022. URL `https://arxiv.org/abs/2110.09813`. ReLoBRaLo: adaptive loss weighting via softmax over relative loss changes with EMA smoothing.